

vLLM & vLLM-Omni

多模态/全模态推理优化

演讲人：莫梓峰

vLLM committer / vLLM-Omni committer

AiCon

全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon 上海站
全球软件开发大会

大模型正在重新定义软件
Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon 北京站
全球人工智能开发与应用大会

探索 AI 应用边界
Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

目录

01

Multi-modal inference in vLLM

02

Omni-modal inference in vLLM-
Omni

03

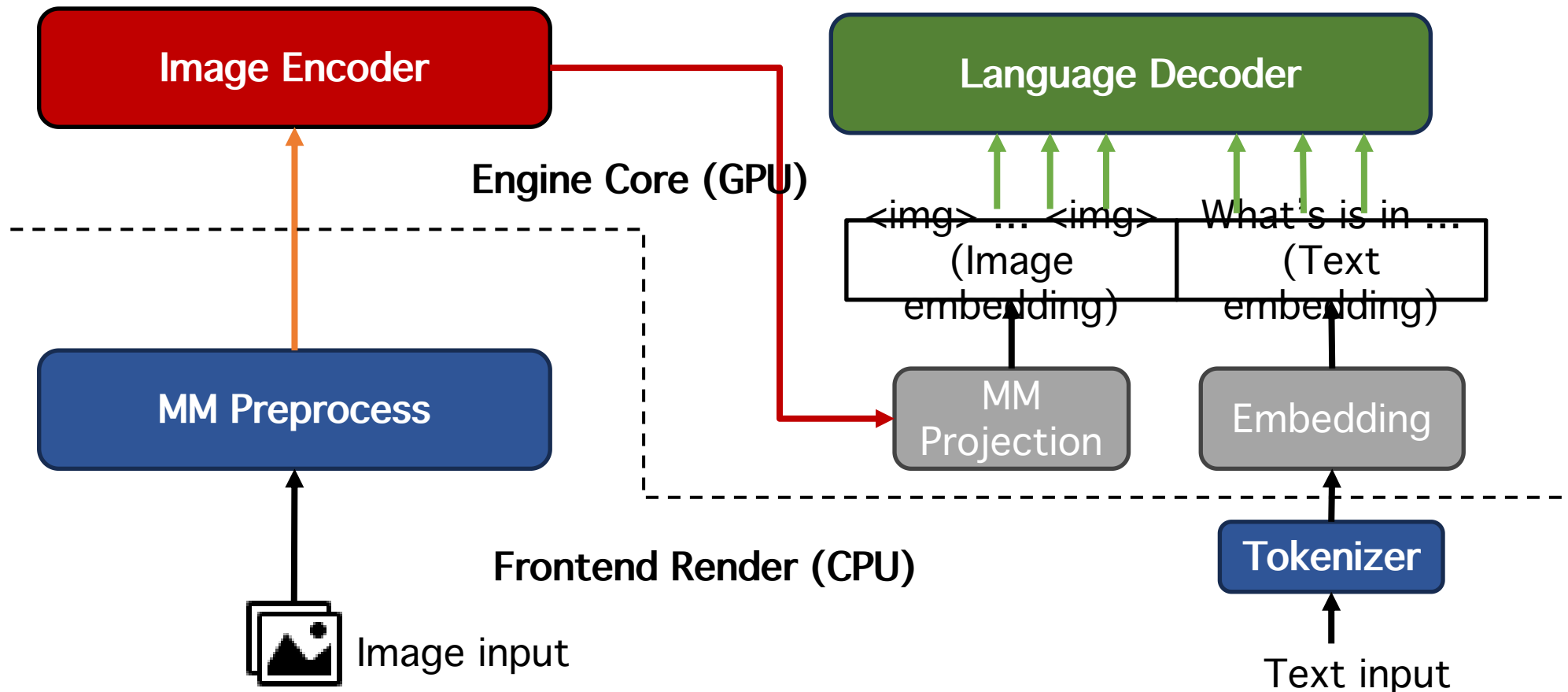
01

Multi-modal inference in vLLM

Scheduling · cache · optimizations

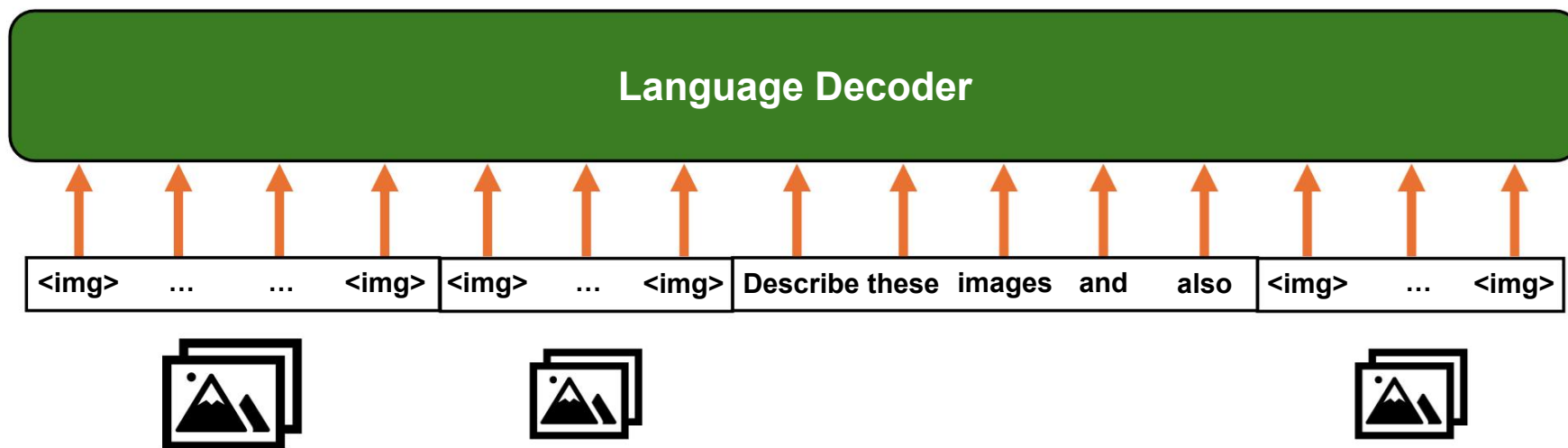
Multi-Modal Inference Pipeline

- Additional execution stage: **Multi-Modal Encoder**
- **Decoder prefill** depends on the outputs from **Multi-Modal Encoder**
- Significant **CPU -> GPU** data transfer



Multi-Modal vs. Text-Only Inference

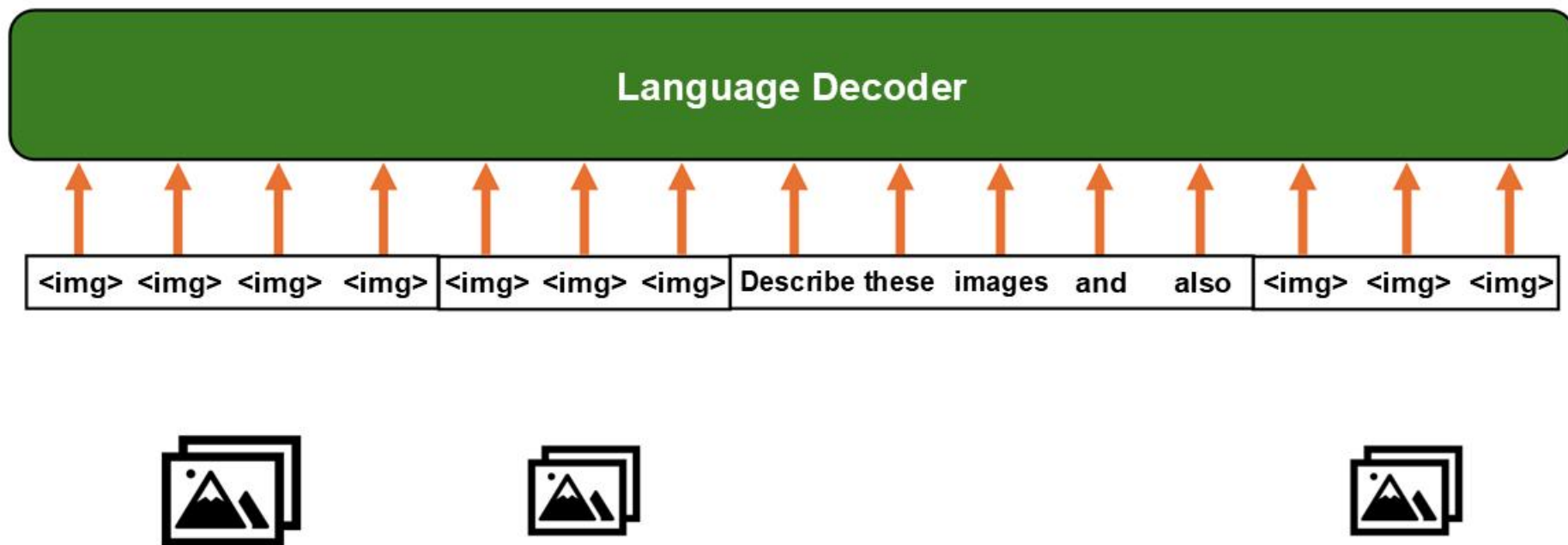
- Text tokens may be interleaved with multi-modal tokens
 - Each multi-modal input may correspond to different number of tokens





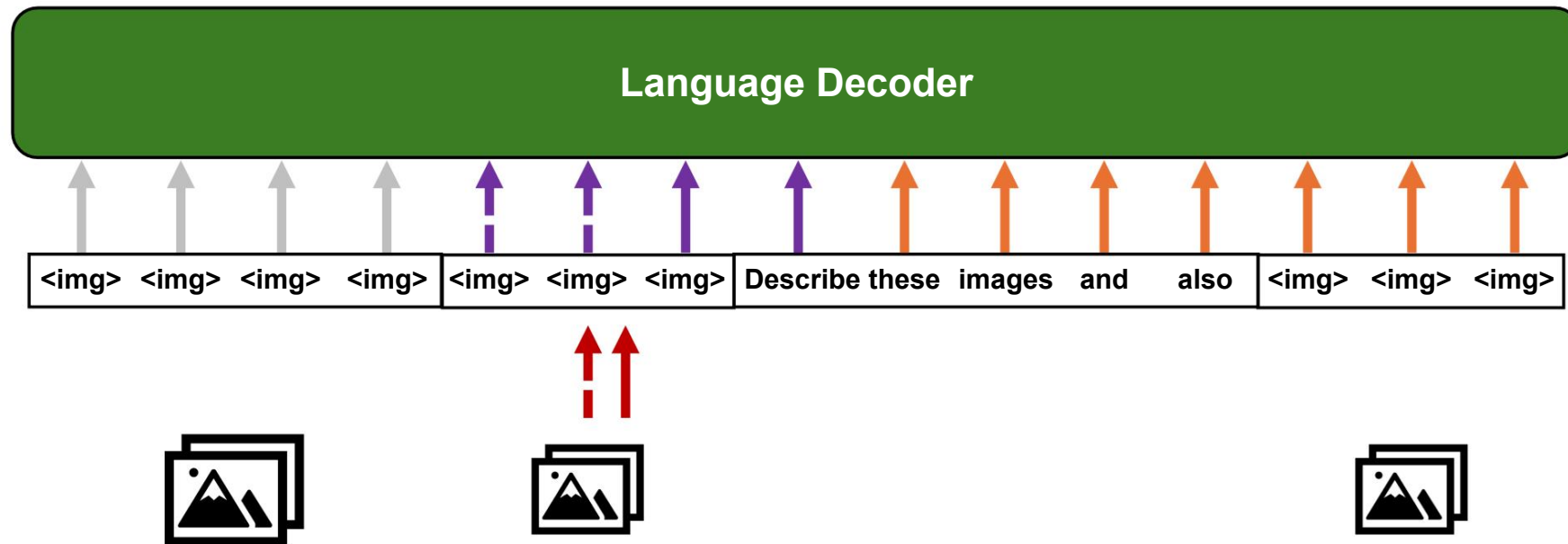
Scheduling Multi-Modal Inputs

- Whenever a multi-modal token is scheduled for decoder prefill, automatically **schedule the corresponding encoder run** as well



Scheduling Multi-Modal Inputs

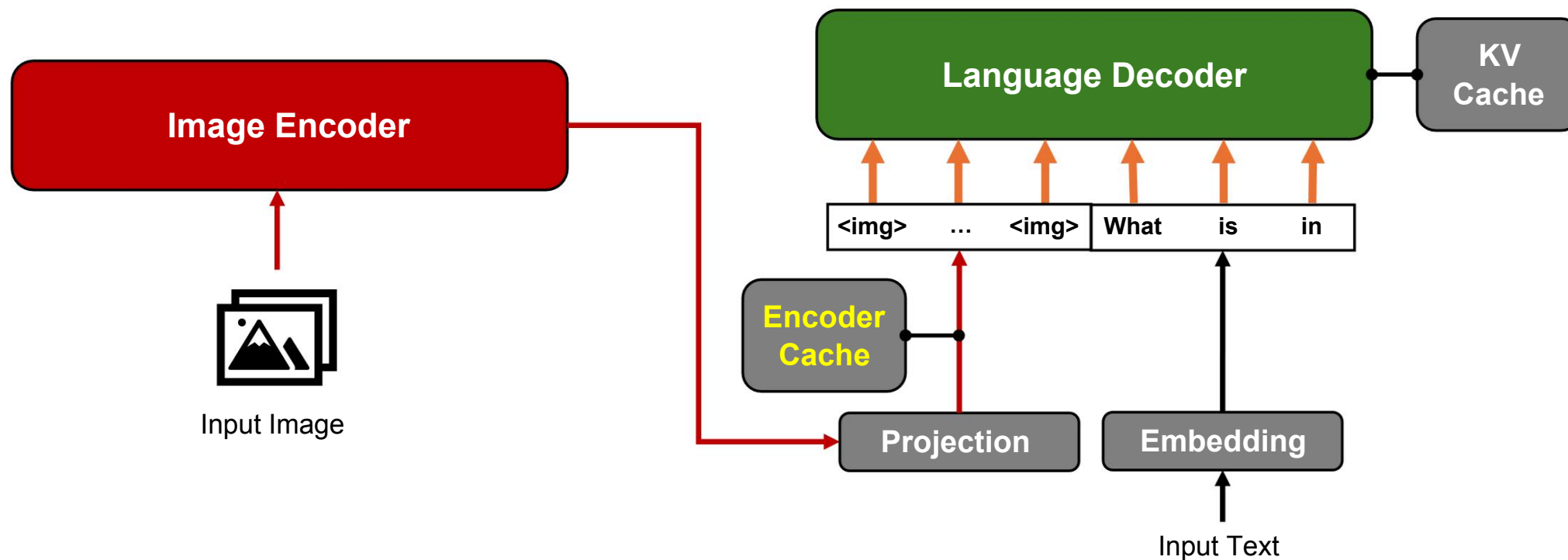
- The same multi-modal input may be **scheduled for encoder run** multiple times if it cannot be fully scheduled in one prefill
 - Example: **Token budget = 2**





Multi-Modal Encoder Cache

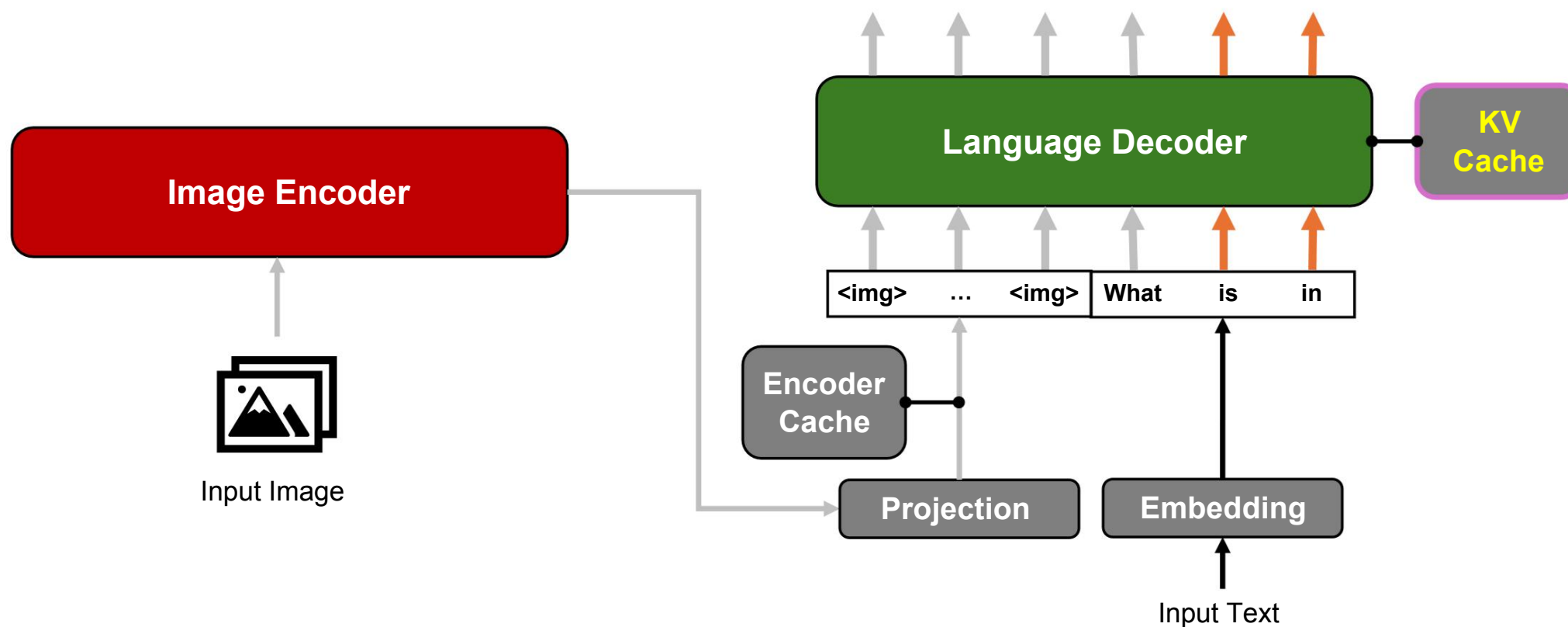
- Store multi-modal embeddings right before merging them with text embeddings
- Mainly accessed during **chunked prefill** for the same request





Multi-Modal Encoder Cache

- **Prefix caching** still works as usual
 - If decoder execution is skipped, no need to execute encoder either





Multi-Modal Encoder Cache

- Can also be used by other requests with the same multi-modal inputs **when prefix caching isn't applicable** because:
 - The system prompt is different, e.g.:
 - Request 1: [**System 1**, **Image A**, Text]
 - Request 2: [**System 2**, **Image A**, Text]



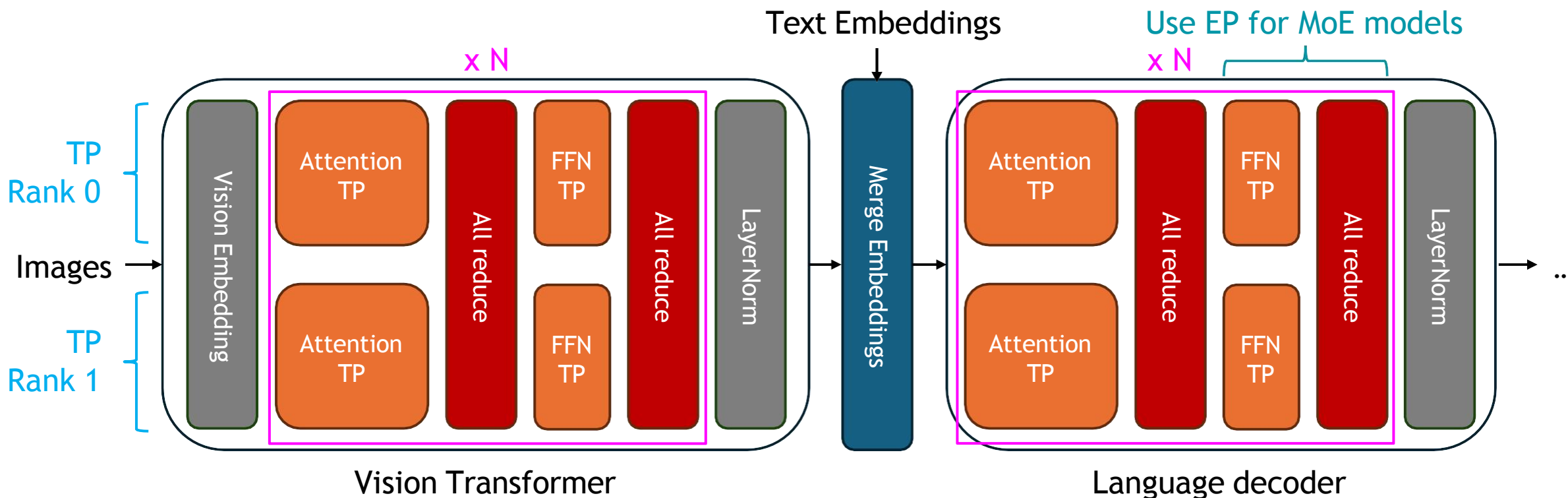
Multi-Modal Encoder Cache

- Can also be used by other requests with the same multi-modal inputs **when prefix caching isn't applicable** because:
 - The system prompt is different, e.g.:
 - Request 1: [System 1, Image A, Text]
 - Request 2: [System 2, Image A, Text]
 - The items come in a different order, e.g.:
 - Request 1: [Image A, Image B, Text]
 - Request 2: [Image B, Image A, Text]



Batch-Level Data Parallelism for the ViT Encoder

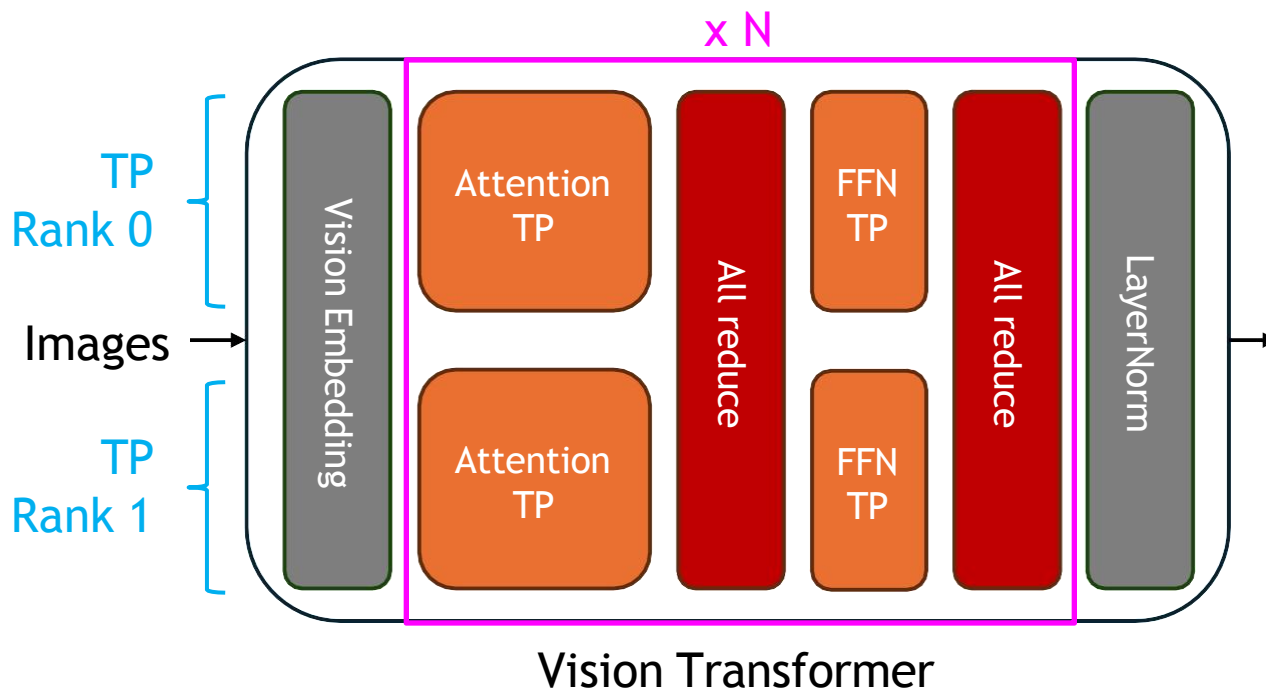
- TP reduces memory requirement by sharding model weights but introduces overhead for inter-process communication





Batch-Level Data Parallelism for the ViT Encoder vLLM

- Multi-modal encoders benefit less from TP because of their **small size (<1B)**



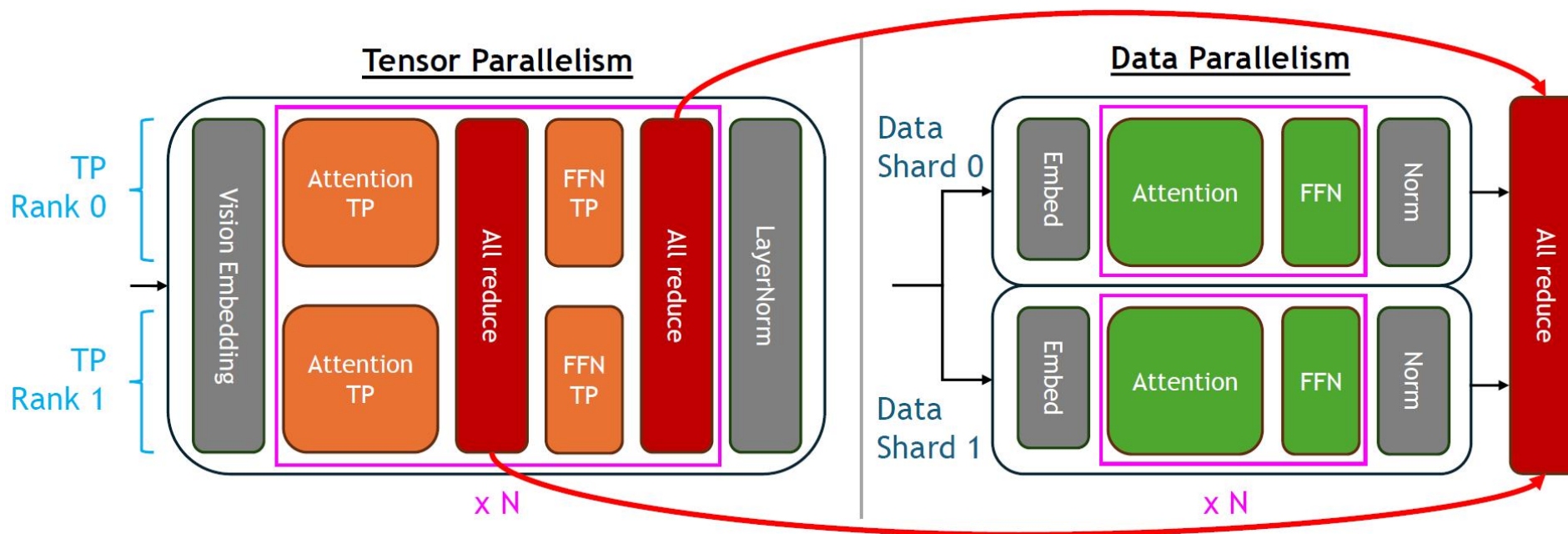
Model Name	Vision Encoder	LLM
Qwen2-VL-2B	675M	1.5B
Qwen2-VL-7B	675M	7.6B
Qwen2-VL-72B	675M	72B

e.g. Qwen2-VL



Batch-Level Data Parallelism for the ViT Encoder

- Shard encoder inputs (instead of weights) across TP ranks
(Decoder is still sharded by weights)
- **Reduces IPC overhead** by moving **all reduce** outside the ViT
- **Small memory increase** due to ViT weights not being sharded





Batch-Level Data Parallelism for the ViT Encoder

- Enable ViT DP using `--mm-encoder-tp-mode="data"`
- Supported on a per-model basis:
 - Llama4 (TP=8: 8.06 -> 8.62 req/s)

PR	#18368
----	--------
 - Step3 (-45% latency in vision encoder)

PR	#22697
----	--------
 - Qwen2-VL (TP=8: 1.79 -> 2.57 req/s)

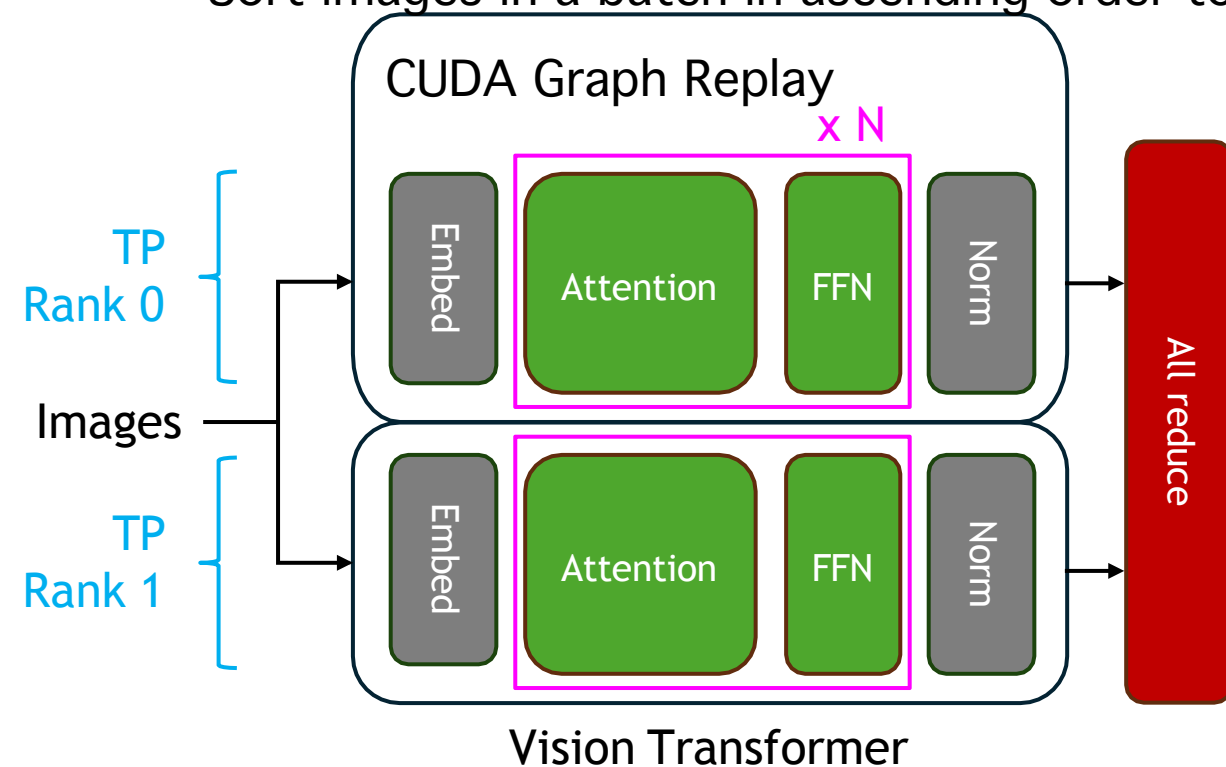
PR	#22742
----	--------
 - MiniCPM-V-4 (TP8: 3.78 -> 5.69 req/s)

PR	#23327
----	--------
 - GLM-4.5V (TP4: 16.25 -> 18.26 req/s)

PR	#22742
----	--------

Full Cuda Graph for the ViT Encoder

- **Budget-based graphs with a maximum batch size:**
In most of cases, Images/video tokens are discrete (eg. 256 tokens/patch)
Capture CUDA graphs at configurable token budgets (e.g., [256, 512, 1024, 2048, 4096]).
- **Greedy bin-packing:**
Sort images in a batch in ascending order to reduce the number of graphs.



Single GPU (Qwen3-VL-30B, 1×GB200, VisionArena-Chat, 3000 prompts):

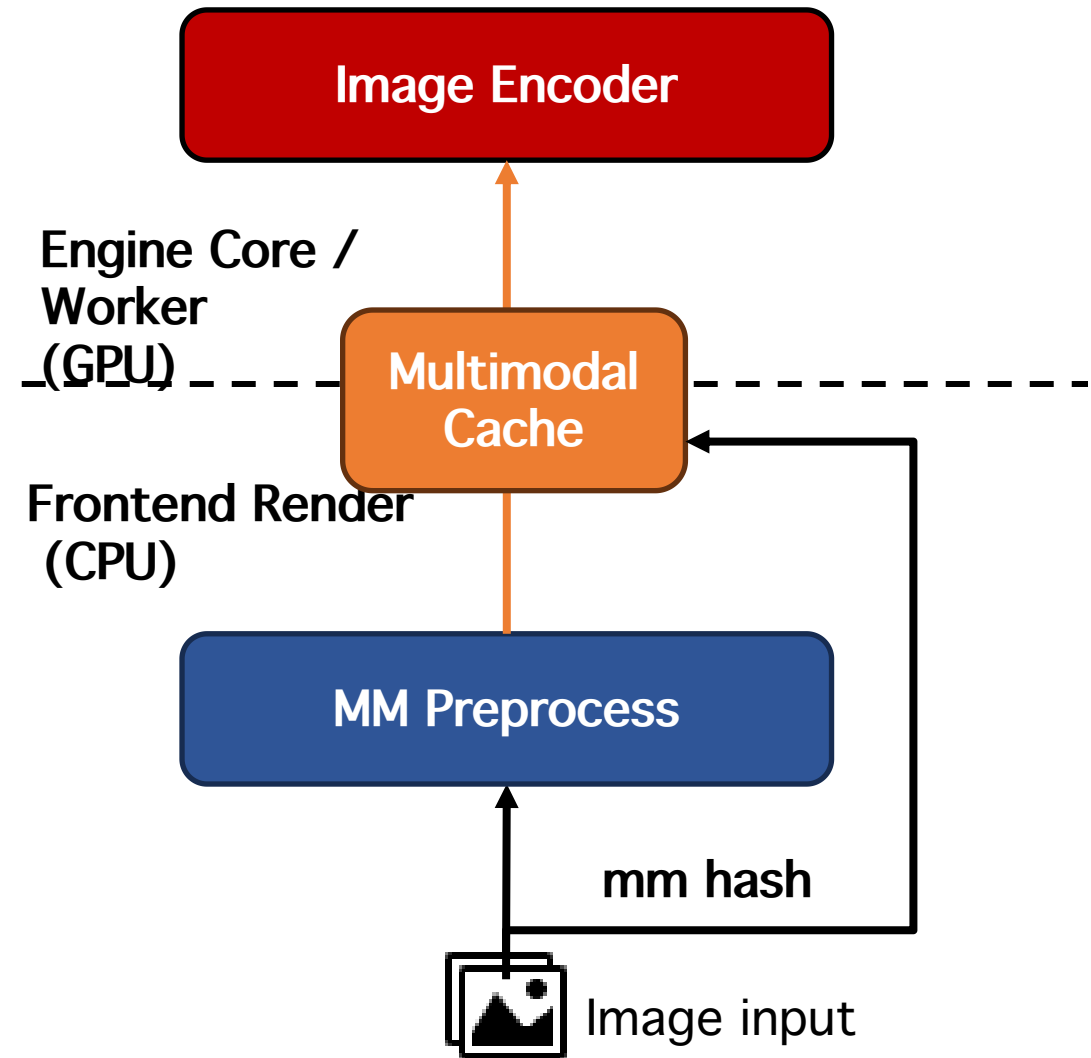
Backend	Mean	P99
FLASH_ATTN	+11.8% (5.13→4.52ms)	+31.6% (9.16→6.26ms)
FLASH_ATTN	+19.6% (5.42→4.36ms)	+40.3% (10.87→6.49ms)

Multi GPU (Qwen3-VL-32B, 4×GB200 TP=4 DP=4, random-mm 20img/req, 1000 prompts):

Backend	Mean	P99
FLASH_ATTN	+18.4% (28.39→23.16ms)	+14.0% (238.78→205.28ms)
FLASHINFER	+44.4% (23.24→12.91ms)	+84.9% (172.41→26.05ms)

Multi-Modal Input Caching

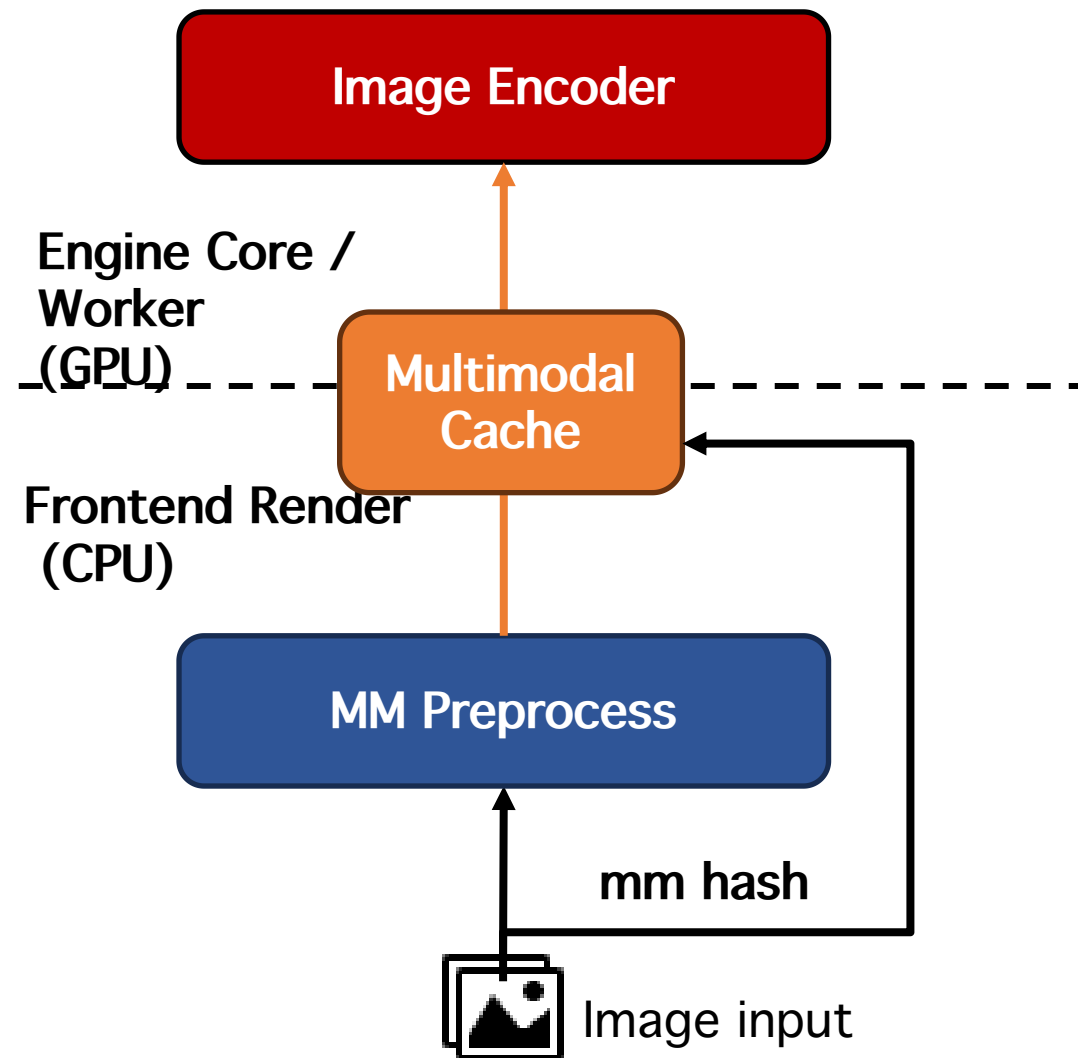
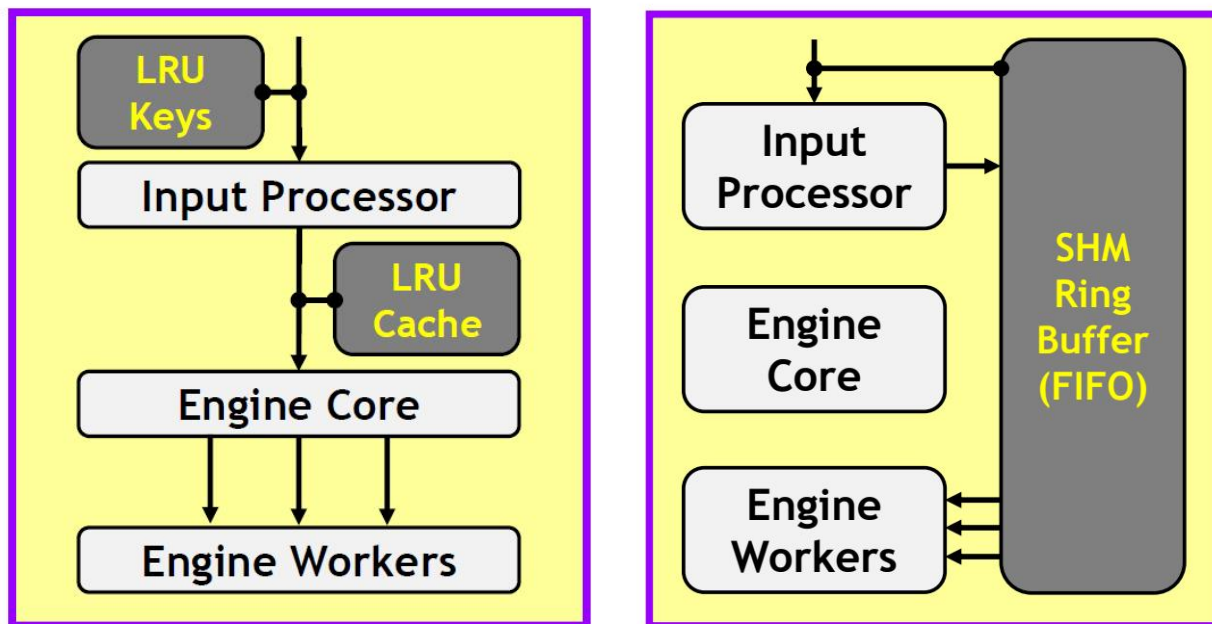
- Each multi modal input is identified by a **unique hash** from raw data / metadata
- If a cache hit would occur at lower layers, pass only its hash to avoid **unnecessary preprocessing / data transfer**



Multi-Modal Input Caching

Cache implementations:

- **LRU:** Cache in the inference engine process, host a copy of keys in frontend
- **SHM:** Cache in shared memory for direct transfer from input process



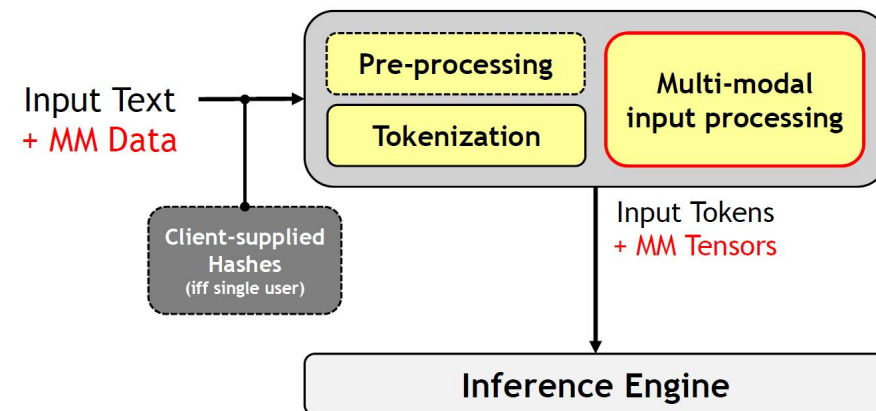
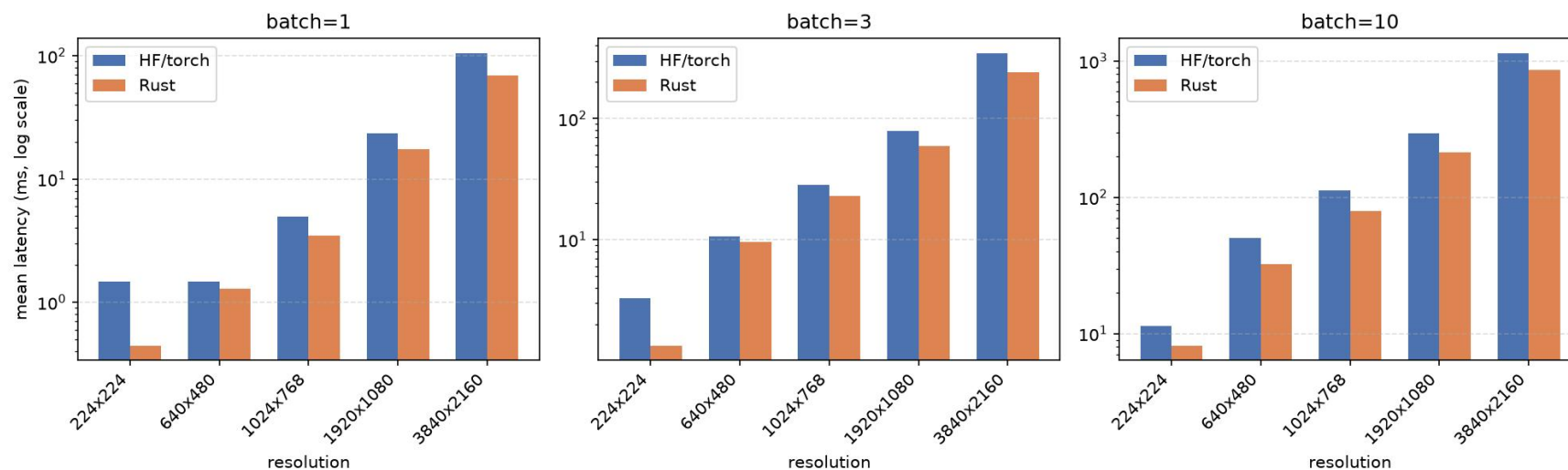
Faster preprocessing with Rust frontend

- Python: image processor is slow because of GIL
- Rust: no GIL limit and we can fuse image preprocessing through low-level ops as well

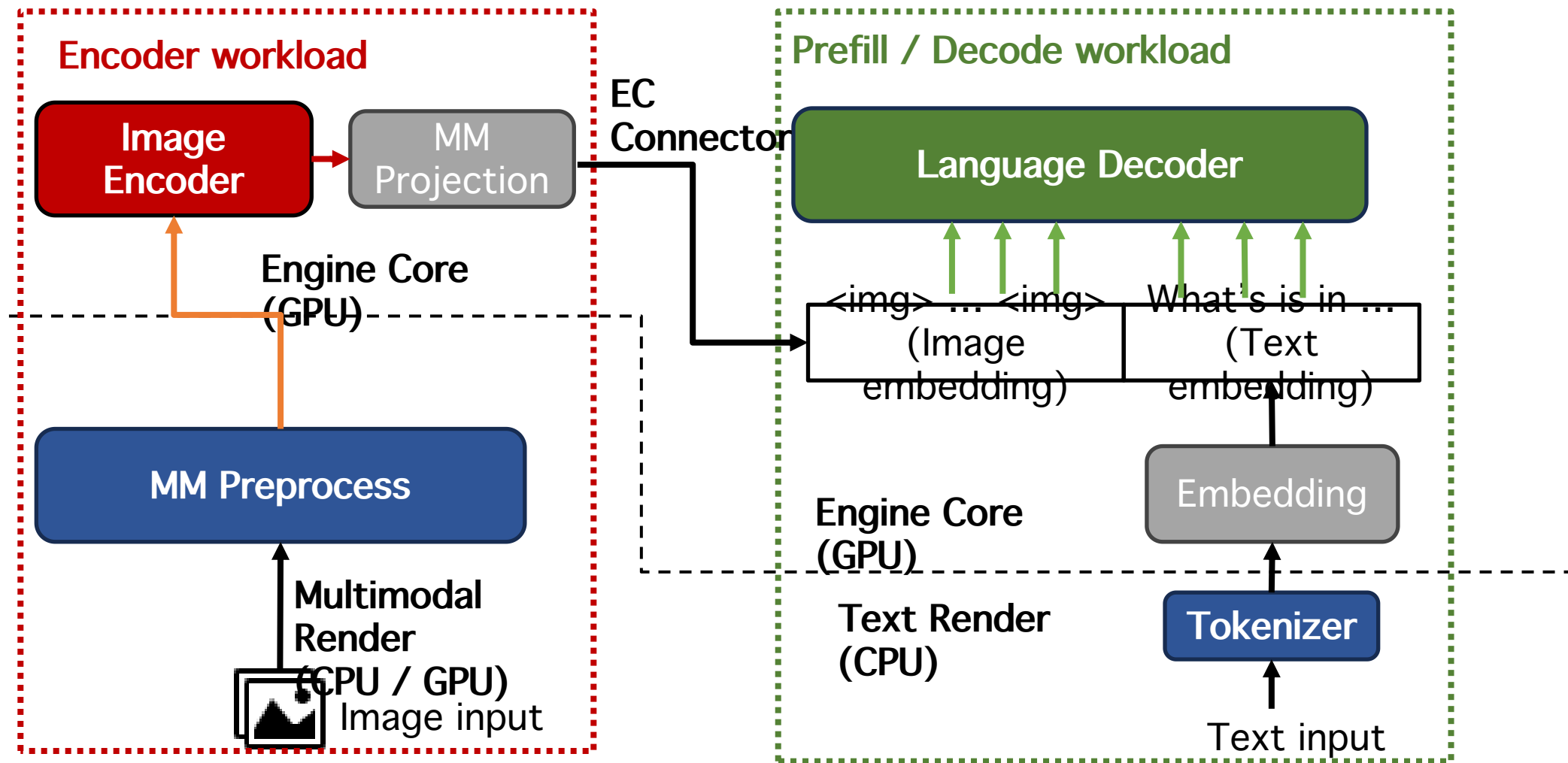
Minimax-M3 with Rust processor on GB200x4

- 1.93 req/s -> 3.18 req/s
- P99 TTFT: 47661ms -> 27332ms

Qwen3-VL image preprocessing: HF/torch vs Rust (CPU)



EPD: Disaggregated Encoder-Prefill/Decode

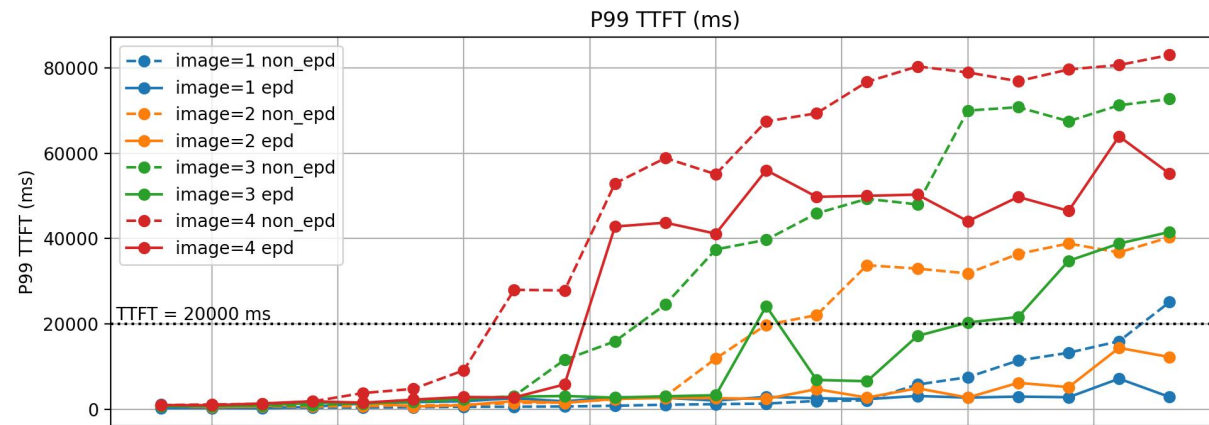
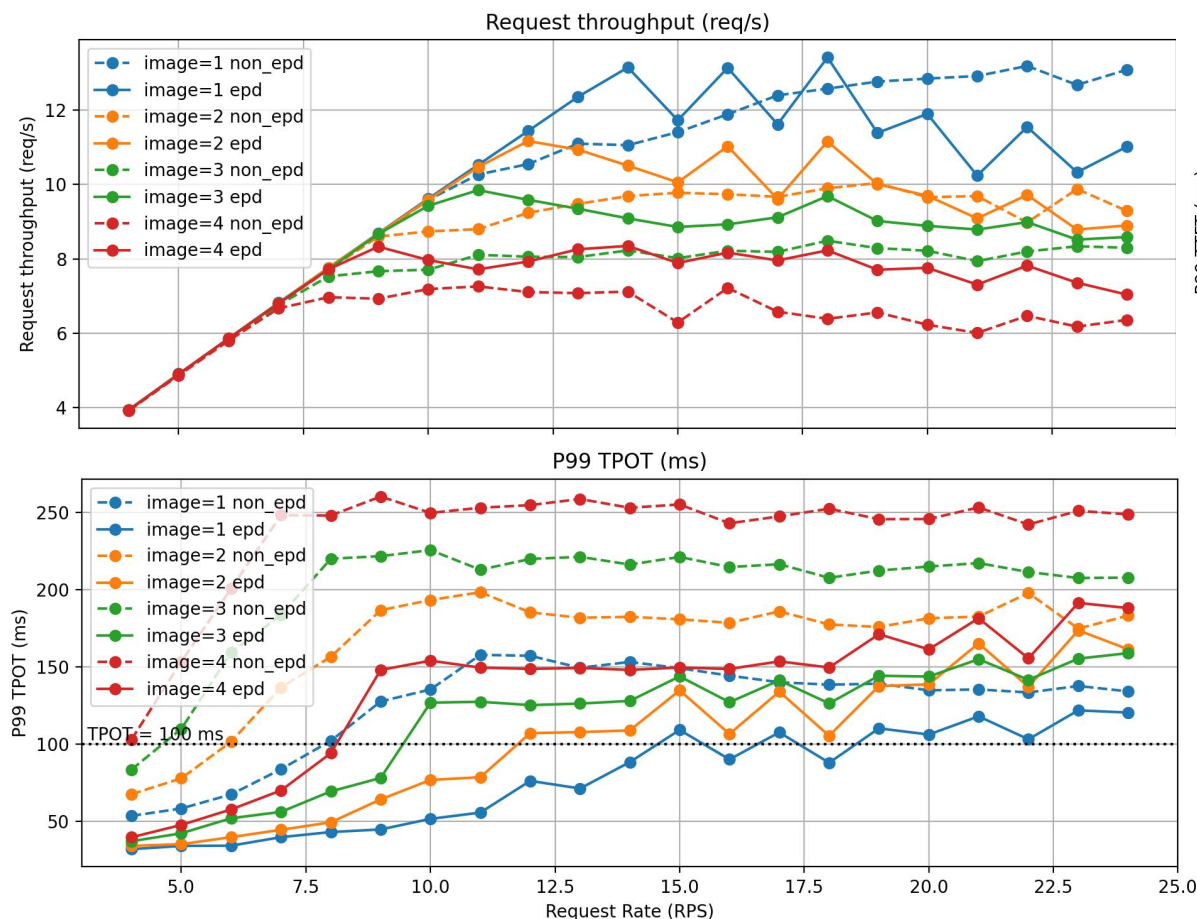


EPD: Disaggregated Encoder-Prefill/Decode

Environment: 4×A100 80G

Model: Qwen3-VL-4B-Instruct, input_len = 2000

Baseline: 1 Encoder + 3 PD instances (1E3PD) vs Data Parallel (4DP)



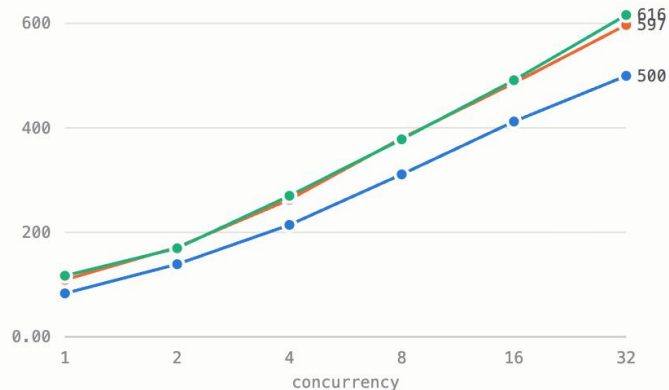
EPD: Unlock GPU multimodal preprocessing

- Use GPU for multimodal preprocess in encoder instance (no kv cache)
- Speed up ~10%
- (WIP) Use GPU's NVDEC for media IO

1 · baseline 2 · no-redundant 3 · no-redundant + gpu preprocess

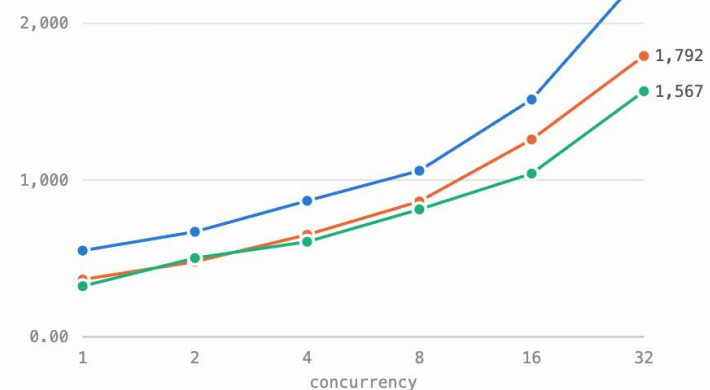
Output token throughput

tok/s · higher is better



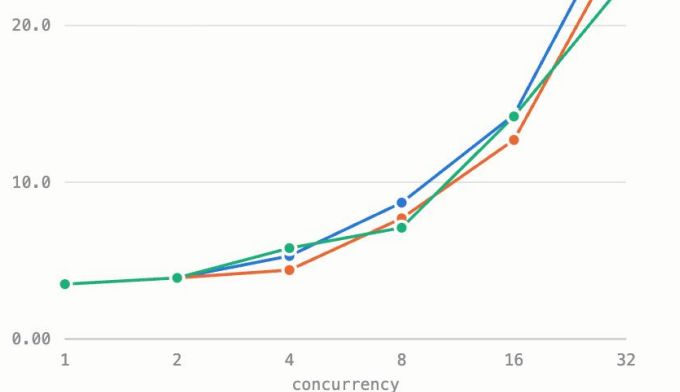
Time to first token (p50)

ms · lower is better



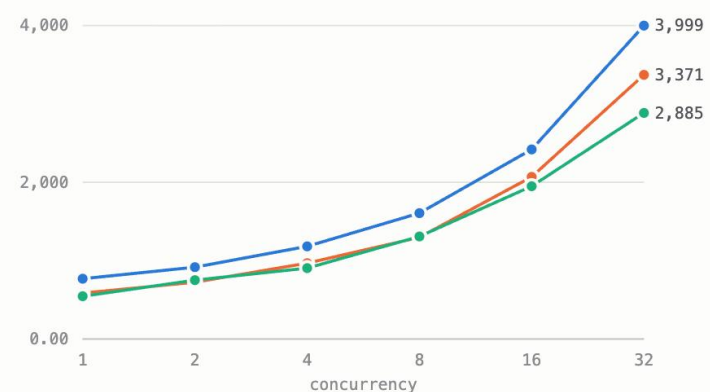
Time per output token (p50)

ms · lower is better



End-to-end latency (p50)

ms · lower is better



Summary

Area	Method	Recommendations
Model Execution	Multi-modal Encoder Cache	Increase cache size if workload has many repeated inputs
	Batch-level DP for ViT Encoder	Enable for TP>1 whenever the model supports it
	Encoder Cuda Graph	Enable if memory enough
Input Processing	Multi-modal Input Caching	Use SHM cache for multi-device inference
	Rust frontend processing	Enable it whenever the model supports it
Scaling	EPD	Enable it if traffic has large amount of multimodal inputs

02

vLLM- Omni

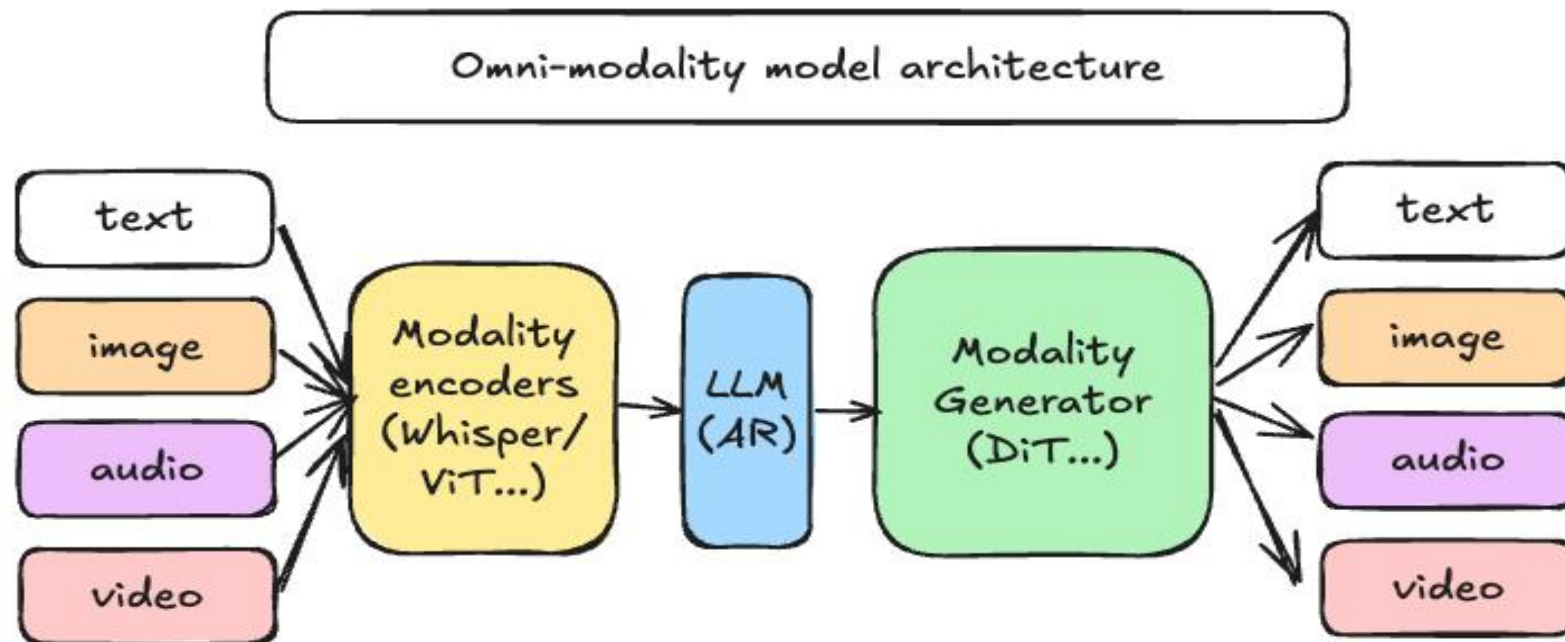
Omni-modality model
serving

Omni-Modality Models: Overview

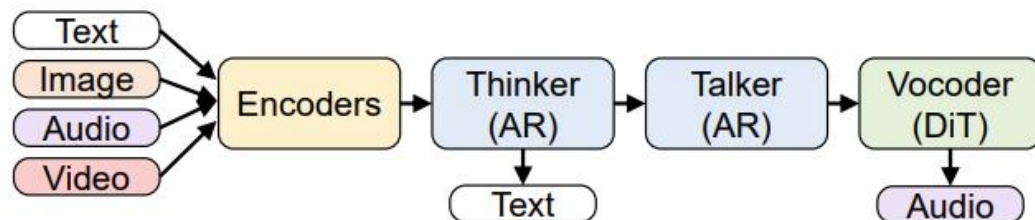
Omni-modality: Text, image, video, and audio data processing

Non-autoregressive Architectures: extend the AR support of vLLM to Diffusion Transformers (DiT) and other parallel generation models

Heterogeneous outputs: from traditional text generation to multimodal outputs



Omni-Modality Models

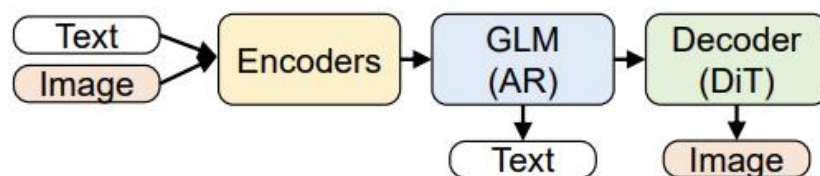


(a) Thinker-Talker (Qwen2.5-Omni)

Backbone: (multi) AR + DiT

Models: Qwen-Omni/Ming-Omni

Tasks: any-to-any

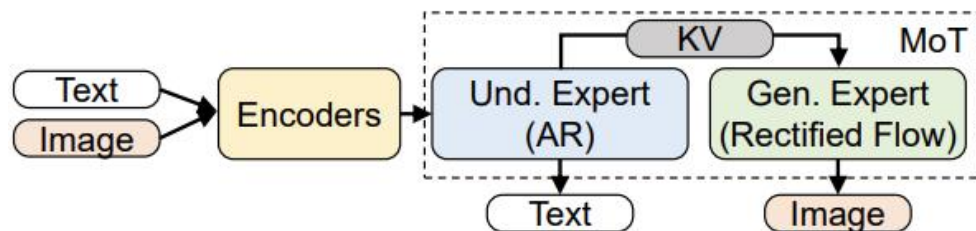


(b) AR + DiT (GLM-Image)

Backbone: AR + DiT

Models: Qwen-Image/GLM-Image

Tasks: t2i, t2v, i2i...



(c) AR + Specialized Generator (BAGEL)

Backbone: AR + Spec. Gen.

Models: BAGEL, Hunyuan Image 3.0

Tasks: t2i, i2i, i2t...

Yin, Peiqi, et al. "vLLM-Omni: Fully Disaggregated Serving for Any-to-Any Multimodal Models." arXiv preprint arXiv:2602.02204 (2026).



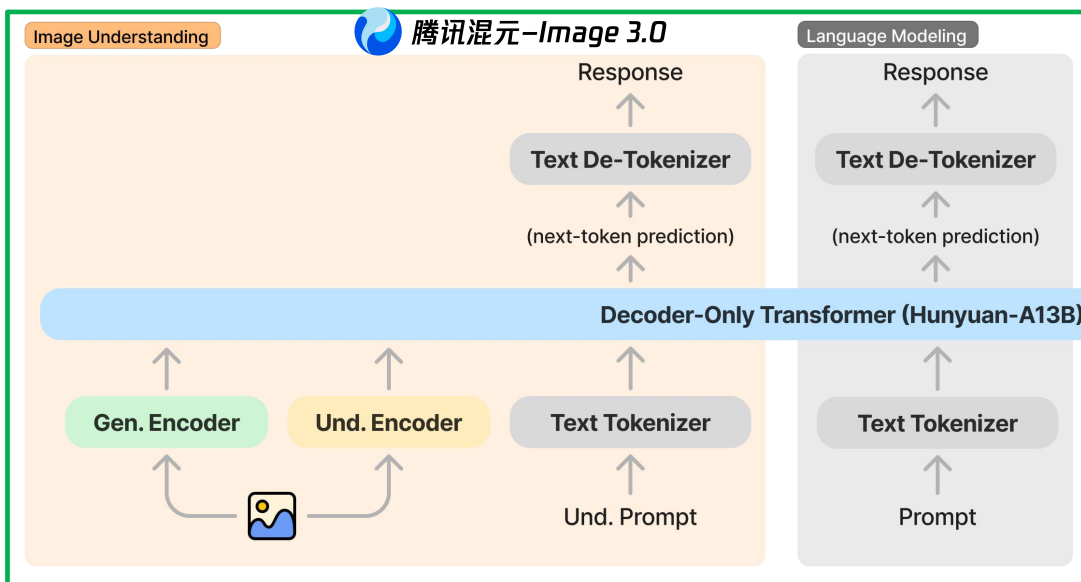
Omni-Modality Models: AR vs. DiT

	AR	DiT
Use cases	Text generation	Multi-modality generation
Generation process	Token-by-token KV Cache based	Diffusion step
Bottleneck	Prefill: compute bound Decode: memory bound	Compute bound
Seq length	varied	Fixed
Attention Mask	causal mask	Full mask
parallelism	TP/DP/EP/PP/CP/SP	TP/EP/USP/CFG

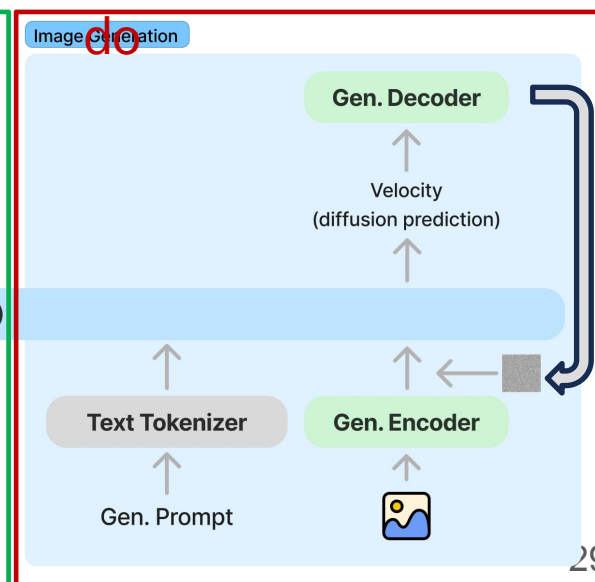
Omni-Modality Models: The rise of omni models

- After Qwen2.5-Omni release, multimodal generation demands increase
- vLLM situation at that time:
 - vLLM engine can optimize AR part, but not DiT
 - vLLM engine is only designed for single AR stage

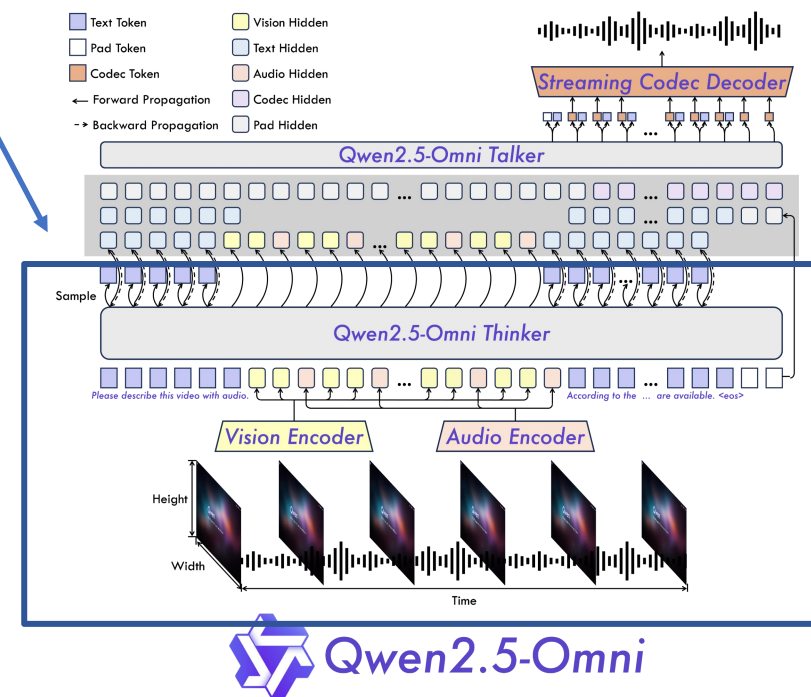
AR – vLLM can do



DiT – vLLM can't do



vLLM serve single AR stage (thinker)

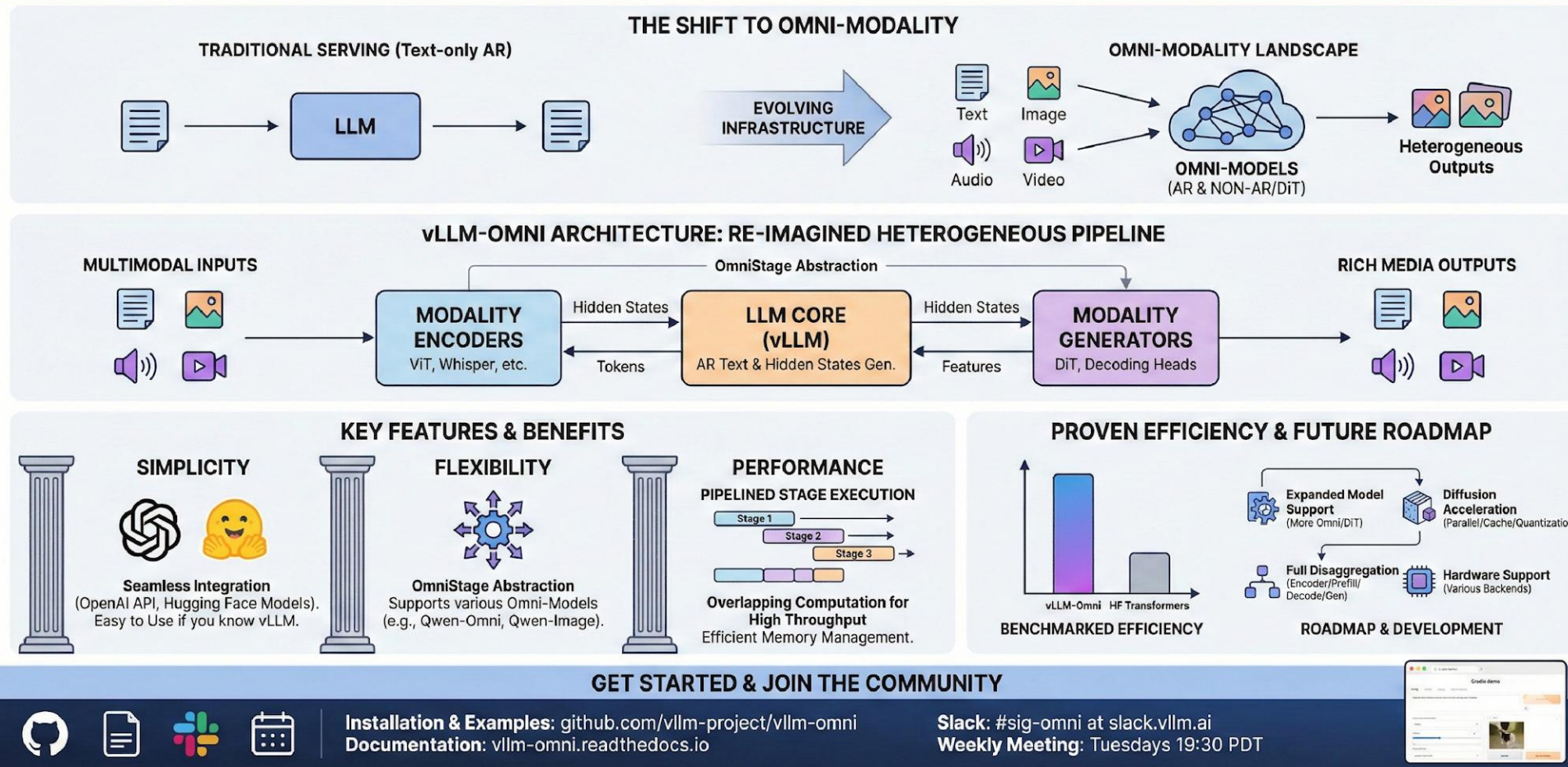




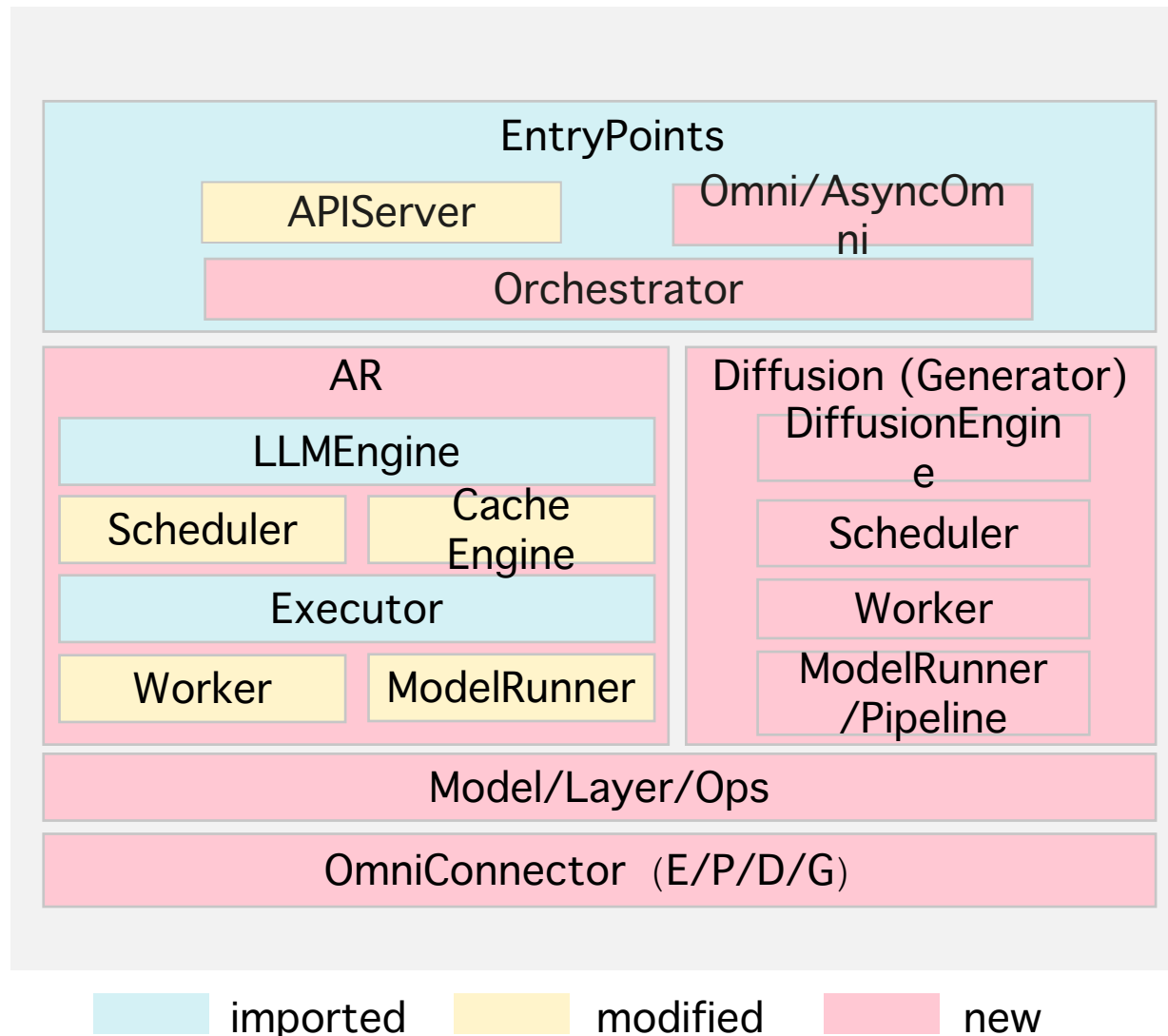
vLLM-Omni Serving Overview



Easy, Fast, and Cheap Omni-Modality Model Serving



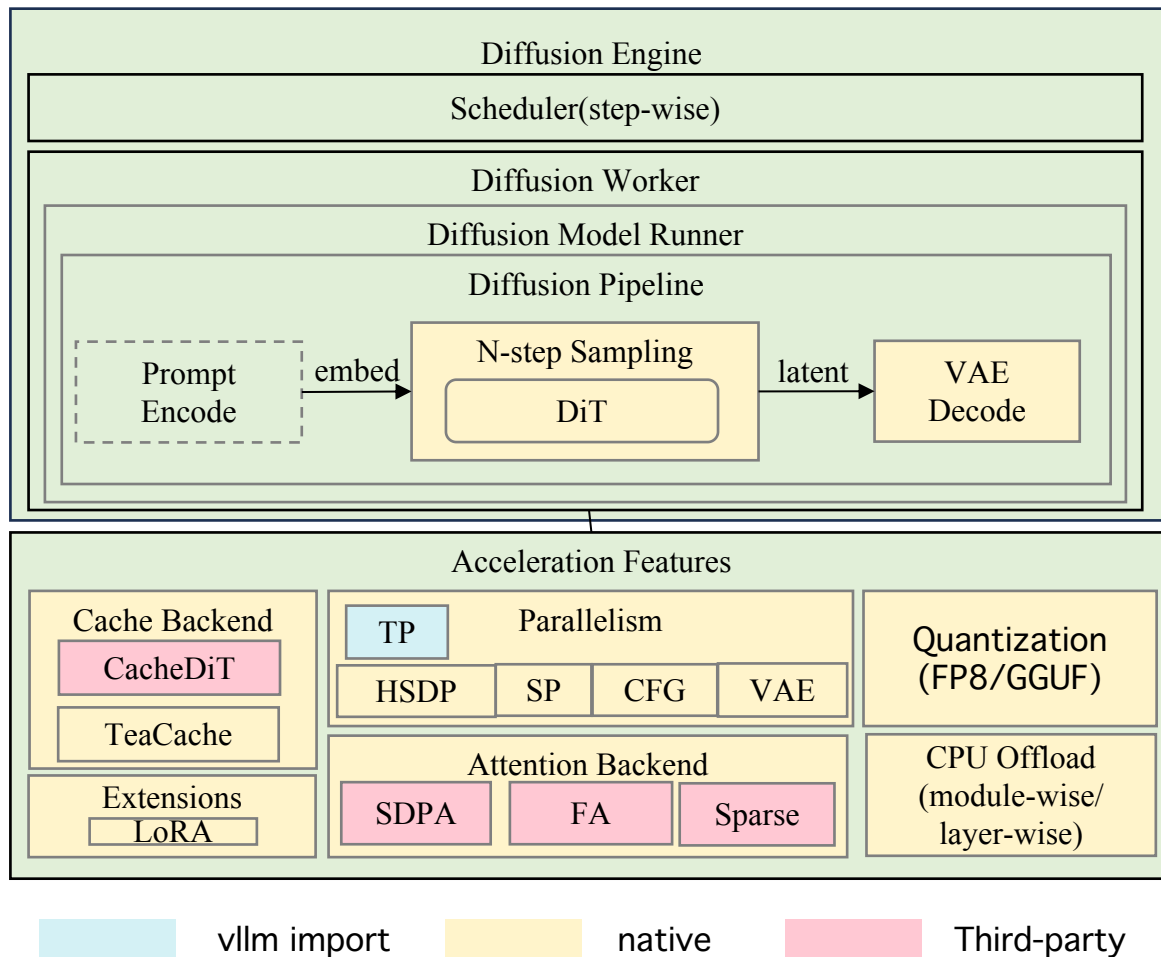
Main Architecture of vLLM-Omni



Main component

- **Entrypoints:** offline/online serving, StageClient abstraction for model stages(AR/DiT). One runtime, multiple interfaces.
- **AR module:** inherited from vLLM(CB/PA/Prefix Cache...) and adapted to the Omni-modality model
- **Diffusion module:** implemented natively and optimized by acceleration components
- **Model/Layer/ops:** parallelism, quantization, attention...
- **OmniConnector:** natively supports E/P/D/G disaggregation

Diffusion Module Design



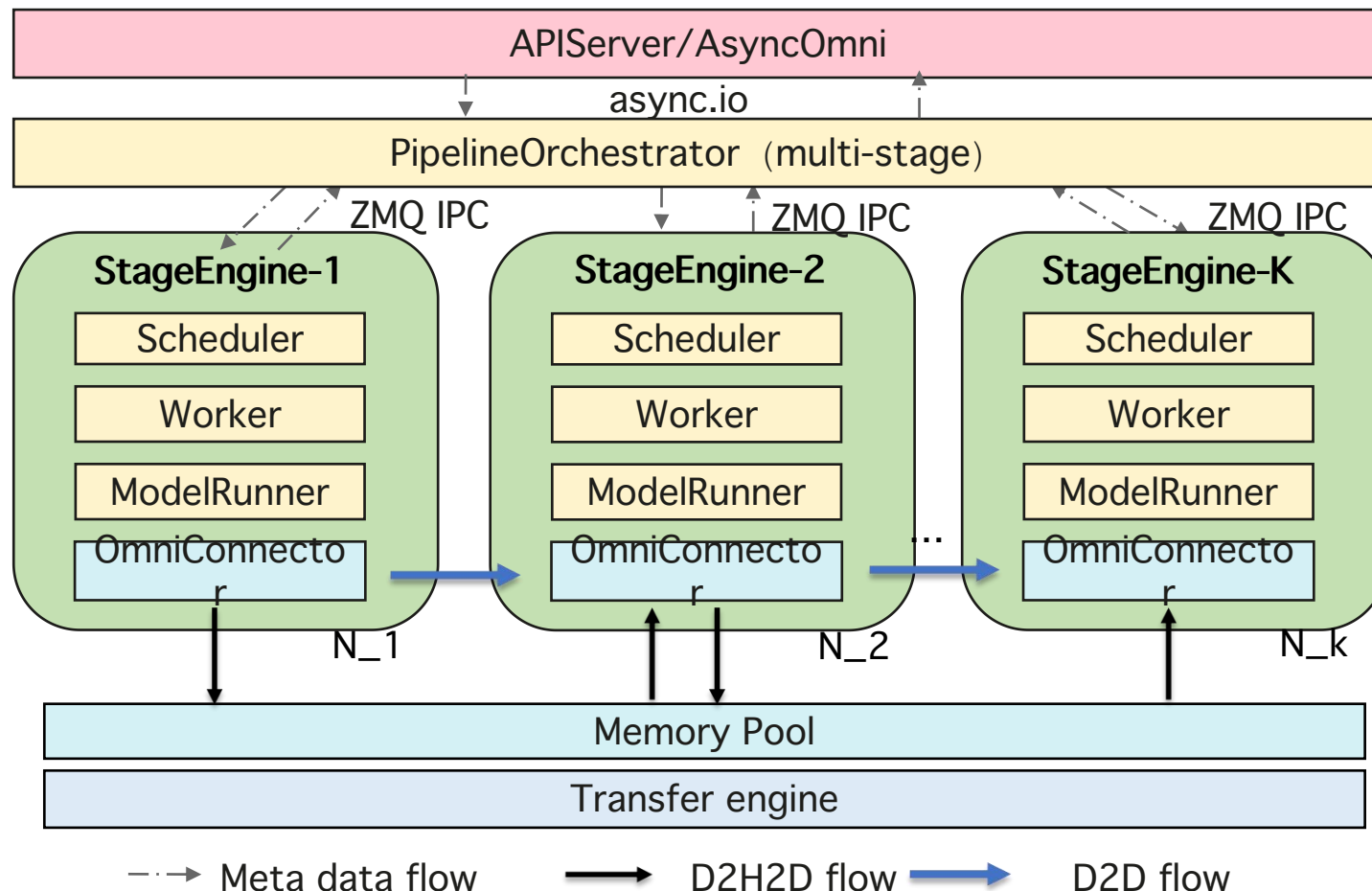
Diffusion Core

- Natively implemented and optimized by configuring the acceleration layer
- Aligned with the AR arch with model_runner abstraction
- *Encoders are disaggregated to run on vllm engine for higher throughput in large-scale deployment*
- *Support step-wise scheduler with continuous batch for higher SLA*
- *Connect with AR module to speed up AR+DiT model inference with OmniConnector*

Acceleration Features

- Cache backend: Cache-DiT, TeaCache...
- Parallelism: TP/HSDP/SP/CFG/VAE...
- Attention: interface abstraction for third-party integration(FA/SAGE/MindIE-SD...)
- Quantization: FP8/GGUF...
- CPU Offload: module-wise/layer-wise
- Extensions: LoRA

Natively Disaggregated Serving

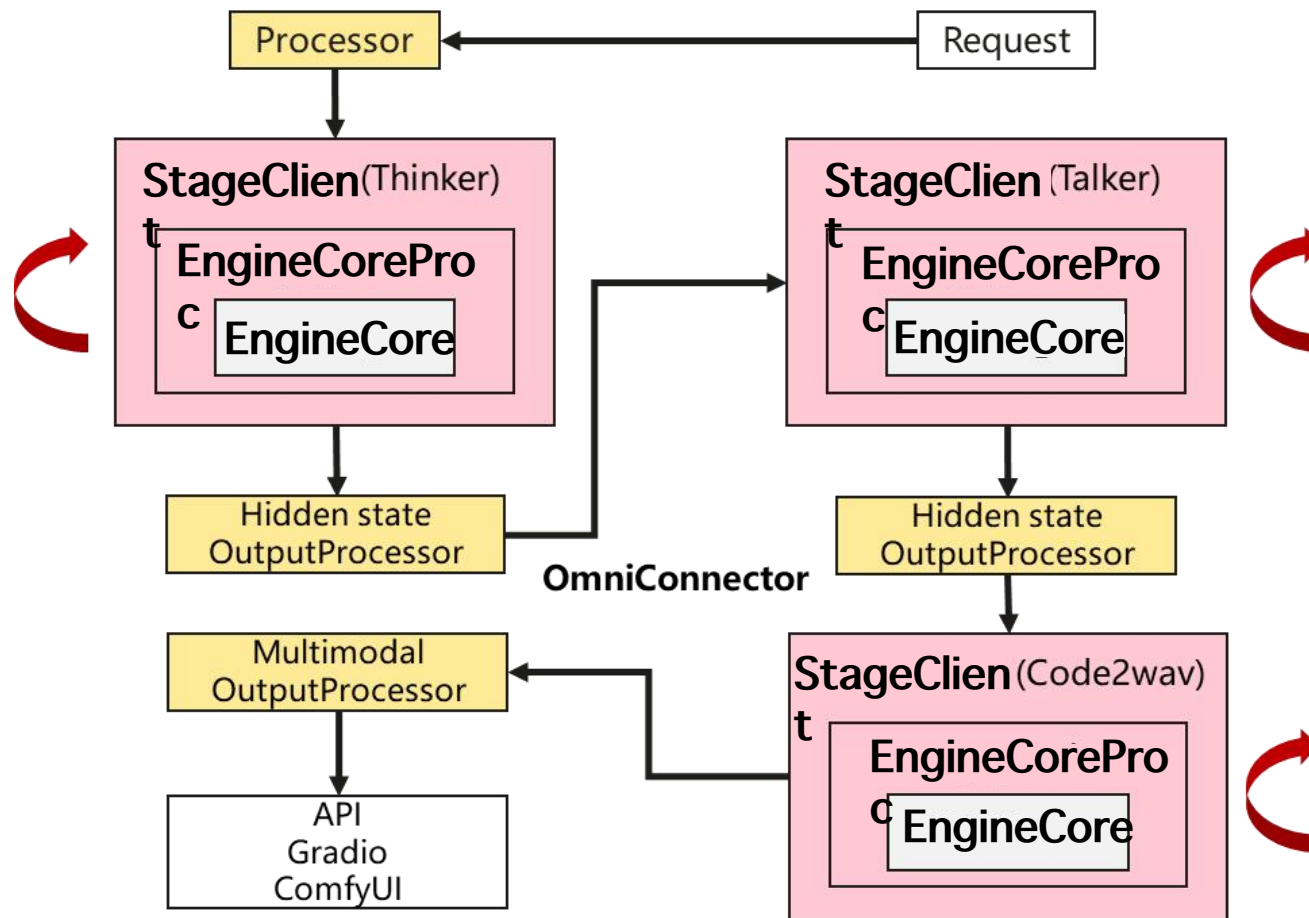
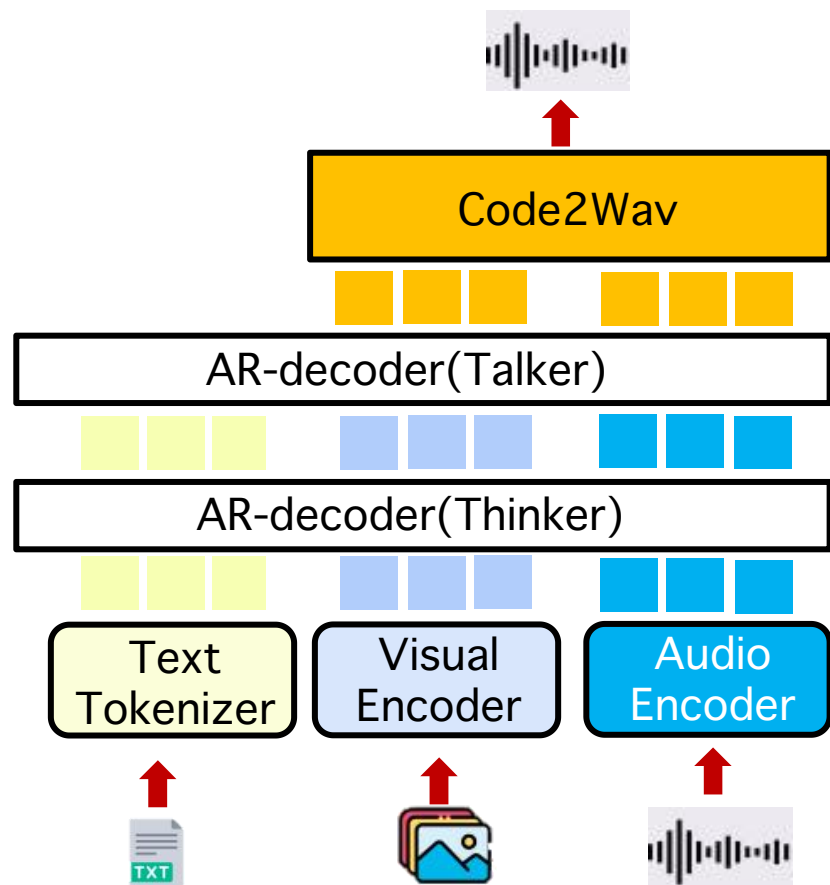


- **Standardized Unified Abstraction:** A generalized interface that handles heterogeneous data (Text, Image, Audio).
- **Control & Data Plane Decoupling:** Metadata travels via lightweight control signals, while heavy payloads are offloaded to high-performance data planes.
- **Hybrid Backend Support:** Native support for Shared Memory (SHM) and Mooncake for distributed transfers.
- **Disaggregated Multi-Modal Execution:** Enables seamless communication between decoupled stages.
- **Multi-Instance Scaling:** Supports multiple instances for each omni stage, enabling elastic deployment and efficient load distribution across distributed clusters.

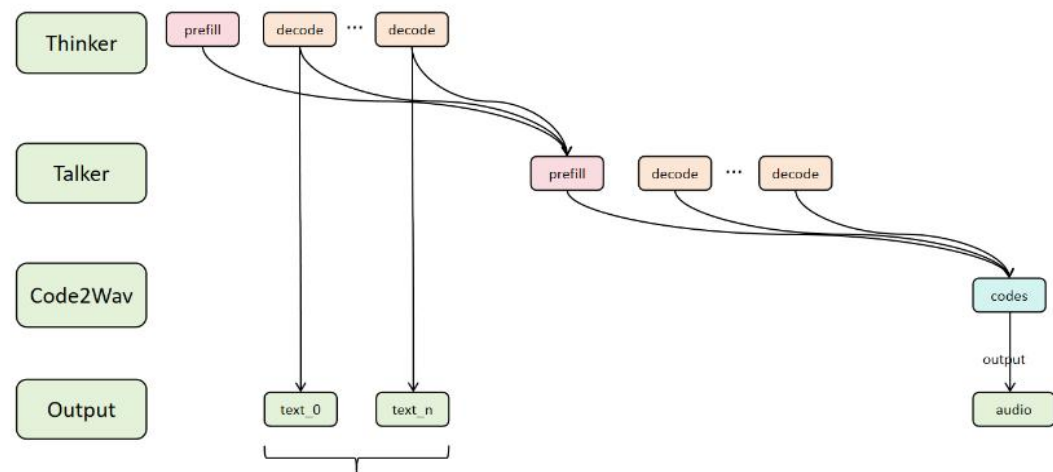


When Requests Arrive

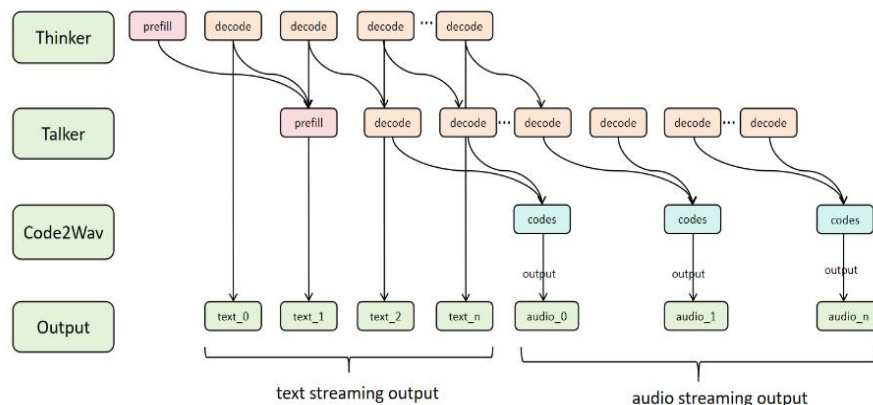
Example: Qwen3-Omni



Async Chunk & Audio Streaming Output



- **Pipeline Between Stages:** Asynchronous chunked computation and communication across stages
- **Audio Streaming Output:** The waveform is output immediately after Talker generates each token.
- **API Server Support:** OpenAI v1/chat/completions with “stream” argument

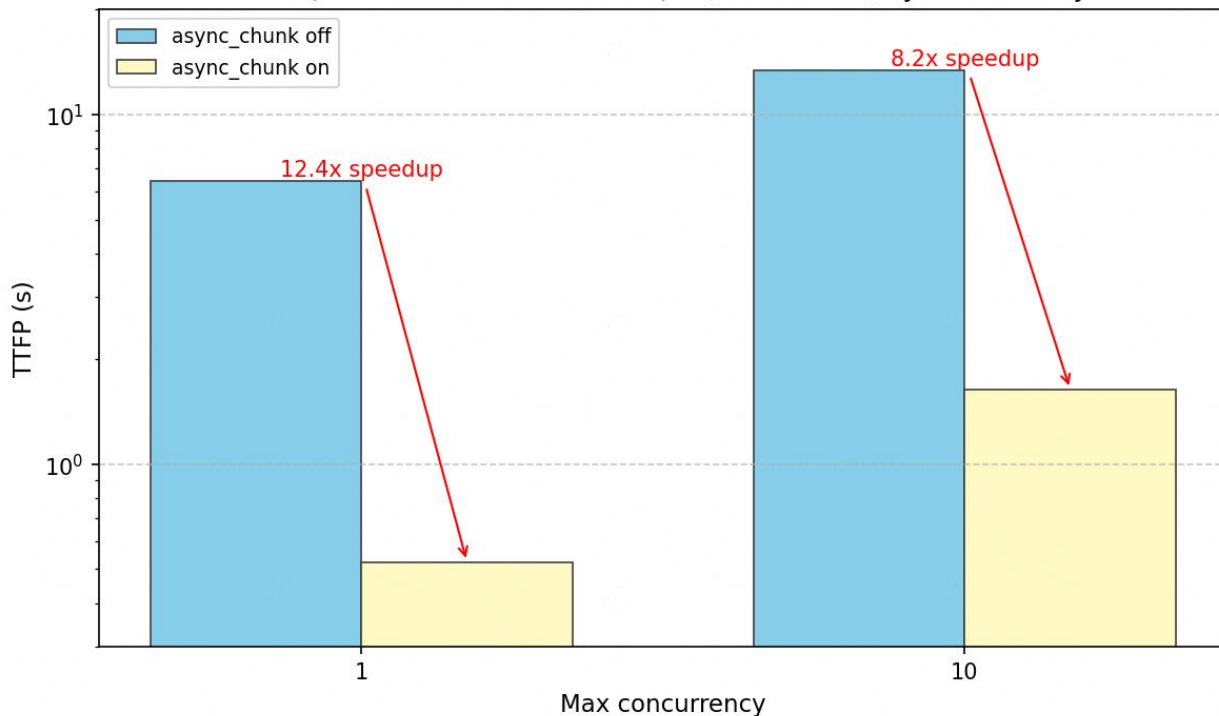


Streaming generation

Async Chunk & Audio Streaming Output

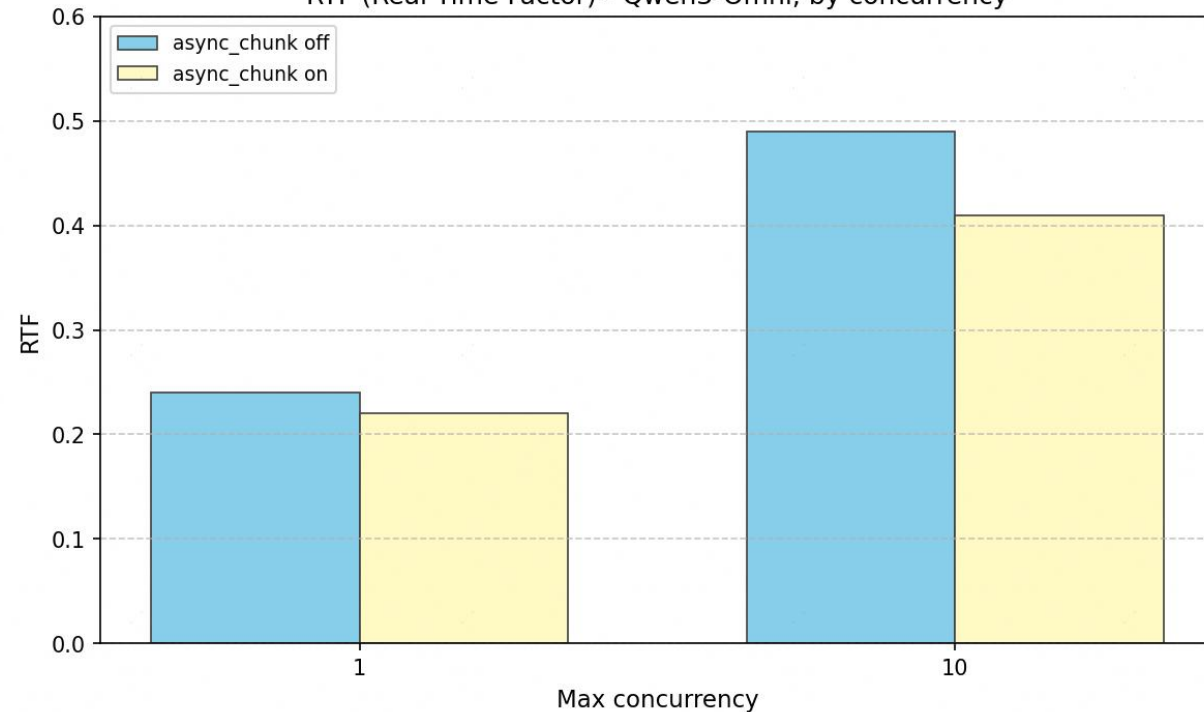
- Qwen3-Omni

TTFP (Time to First Audio Packet) - Qwen3-Omni, by concurrency



1 / 10 concurrency: 12.4x / 8.2x improves

RTF (Real Time Factor) - Qwen3-Omni, by concurrency



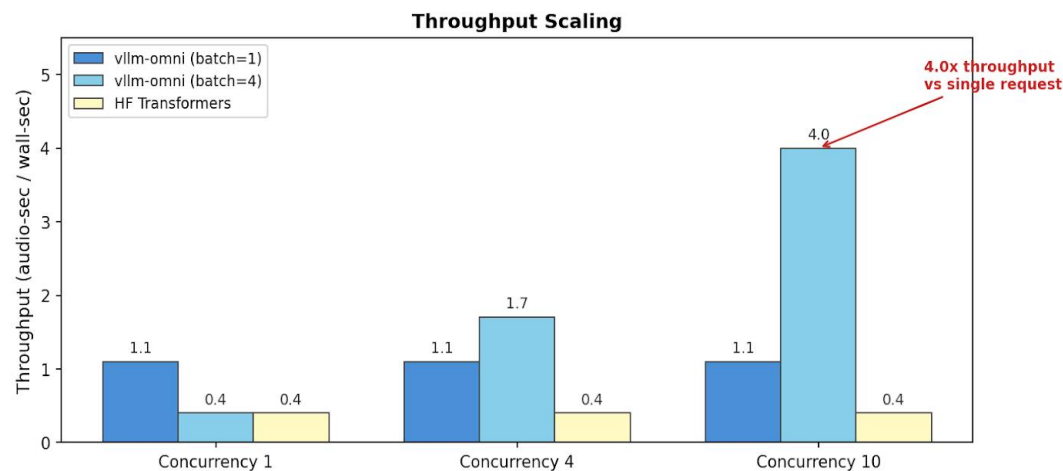
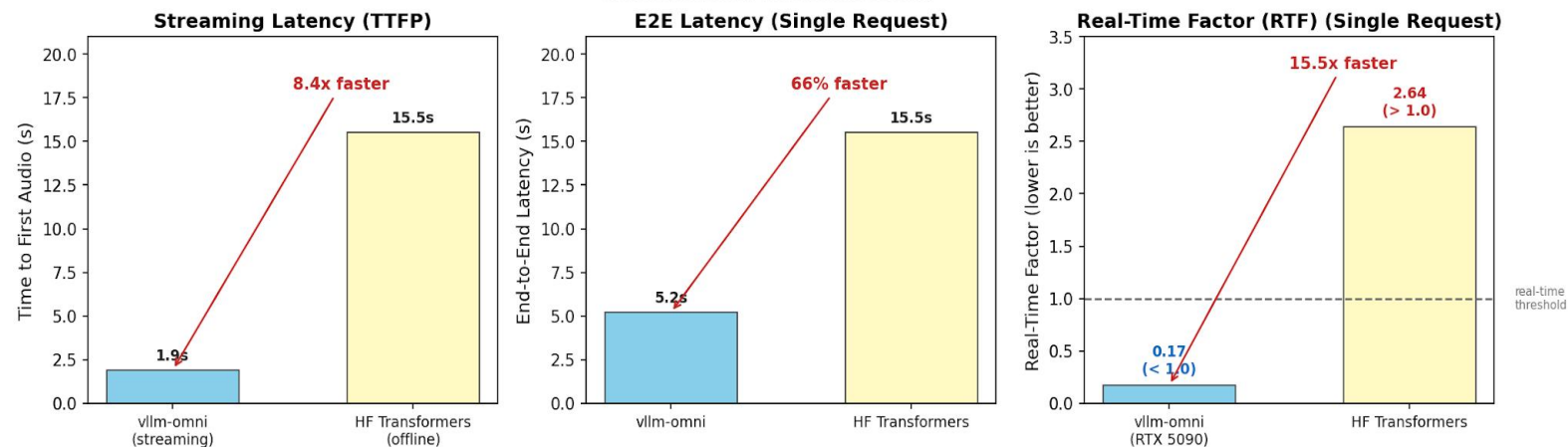
1 / 10 concurrency: 1.1 x / 1.2 x improves

Async Chunk & Audio Streaming Output

- Qwen3-TTS

Qwen3-TTS 1.7B: vllm-omni vs HuggingFace Transformers

Model: Qwen3-TTS-1.7B-CustomVoice | GPU: NVIDIA H200 NVL | 50 requests per test
Without specification, H200 is used for testing.



Key Improvements

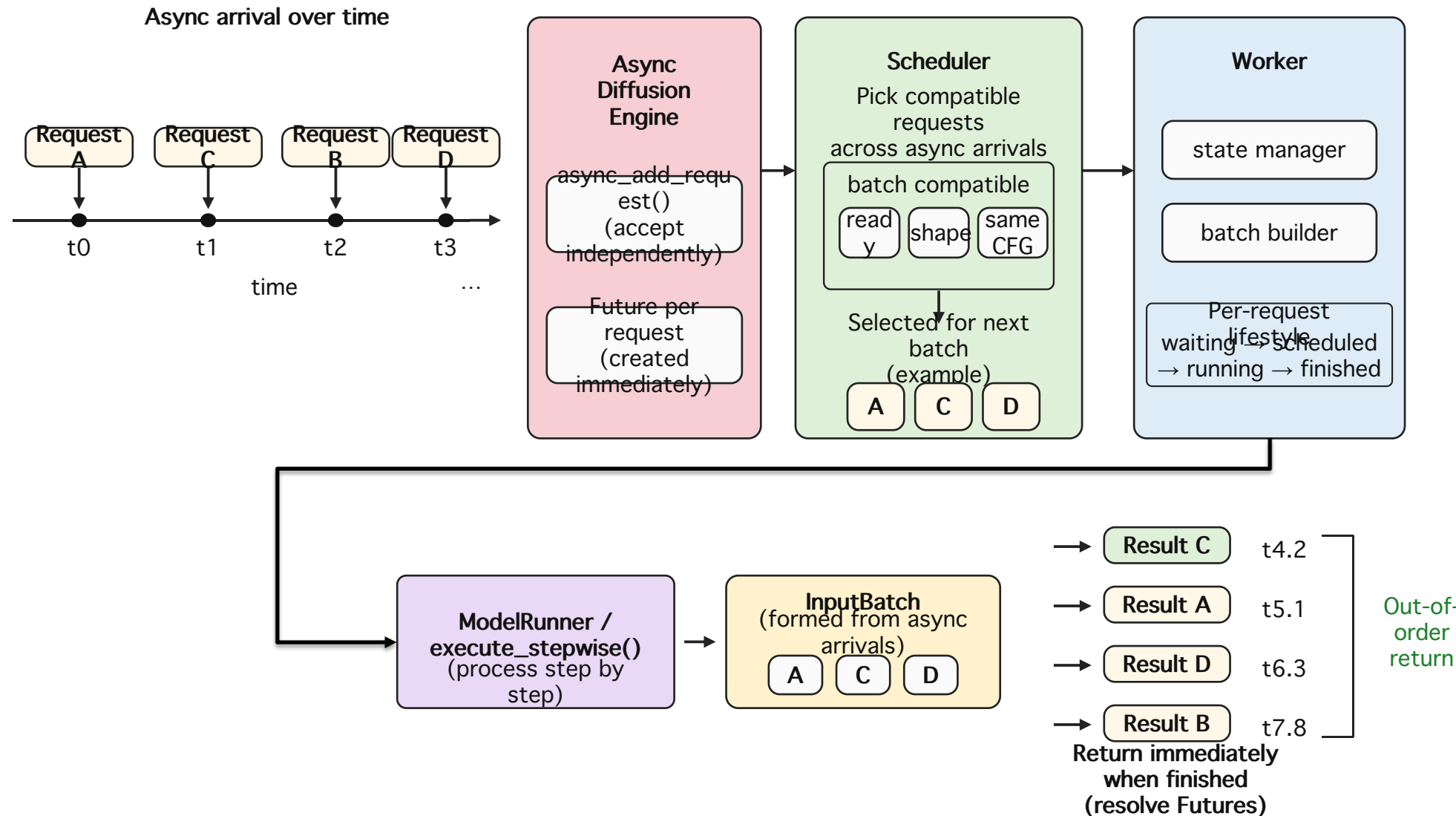
8.4x
faster time-to-first-audio
vs HF (streaming)

66%
lower E2E latency
at single concurrency

RTF 0.17
real-time capable
(HF: 2.64)

4.0x
throughput scaling
with batch=4

Diffusion Dynamic Batching



Diffusion Dynamic Batching

- Latency reduces about 10% on A100

```
===== Serving Benchmark Result =====
Backend:                                vllm-omni
Model:                                  Qwen/Qwen-Image
Dataset:                                vbench
Task:                                   t2i
-----
Benchmark duration (s):                  2452.97
Request rate:                            inf
Max request concurrency:                  8
Successful requests:                     100/100
-----
Request throughput (req/s):               0.04
Latency Mean (s):                        189.3709
Latency Median (s):                      196.1949
Latency P99 (s):                         196.2799
Latency P95 (s):                         196.2570
-----
Stage Durations Mean (s):
  queue_wait_ms:                         0.6736
  stage_0_gen_ms:                        188944.5911
=====
```



```
===== Serving Benchmark Result =====
Backend:                                vllm-omni
Model:                                  Qwen/Qwen-Image
Dataset:                                vbench
Task:                                   t2i
-----
Benchmark duration (s):                  2275.14
Request rate:                            inf
Max request concurrency:                  8
Successful requests:                     100/100
-----
Request throughput (req/s):               0.04
Latency Mean (s):                        178.4276
Latency Median (s):                      183.5964
Latency P99 (s):                         183.9953
Latency P95 (s):                         183.9803
-----
Stage Durations Mean (s):
  queue_wait_ms:                         0.8274
  stage_0_gen_ms:                        175764.1154
=====
```

1. Support Omni-Modality model inference & serving
2. Consistent and unified API interface with vLLM
3. Native disaggregated deployment for different model stages
4. Pipeline async streaming for multiple stage engines
5. Native support for diffusion stage acceleration

■ Networking Hour – vLLM

Building the **fastest** and **easiest-to-use**
open-source LLM inference & serving

engine!

 <https://github.com/vllm-project/vllm>

 <https://docs.vllm.ai>

 <https://blog.vllm.ai>

 <https://discuss.vllm.ai/>

 <https://www.zhihu.com/people/vllm-team>

 <https://www.linkedin.com/company/vllm-project>

 <https://slack.vllm.ai>

 https://twitter.com/vllm_project

 <https://opencollective.com/vllm>

 vllm_project

 vllm_project

 vllm-questions@lists.berkeley.edu



Networking Hour – vLLM-Omni

vLLM-Omni



<https://github.com/vllm-project/vllm-omni>



<https://blog.vllm.ai/2025/11/30/vllm-omni.html>



<https://communityinviter.com/apps/vllm-dev/join-vllm-developers-slack>

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

Q&A

github.com/vllm-project

AiCon

全球人工智能开发与应用大会